



UNIVERSITÀ DEGLI STUDI DI SALERNO

Dipartimento di ingegneria dell'informazione ed elettrica e matematica applicata

Corso di laurea magistrale in ingegneria informatica

Anno accademico 2024/2025

High Performance Computing: Final Homework

Studente: Orlando Palma 0622702433

Professore: Francesco Moscato

Sommario

INTRODUZIONE.....	1
CAPITOLO 1: Fondamenti	2
1.1 Suffix Array.....	2
1.2 Algoritmo di Manber-Myers	3
1.3 Algoritmo di Kasai.....	4
1.4 Longest Repeated Substring	5
CAPITOLO 2: Implementazione Sequenziale.....	6
2.1 Architettura e Scelte Implementative	6
2.2 Implementazione dell'Algoritmo di Manber-Myers	7
2.3 Implementazione dell'Algoritmo di Kasai	9
2.4 Ricerca della Longest Repeated Substring	11
2.5 Flusso di Esecuzione	12
CAPITOLO 3: Implementazione Parallela MPI.....	13
3.1 Elaborazione Distribuita e Ordinamento Locale	13
3.2 Raccolta Dati e Preparazione al Merge.....	15
3.3 Merge Iterativo: Tournament Merge.....	17
3.4 Post-Processing Sequenziale	20
3.5 Flusso di Esecuzione	20
CAPITOLO 4: Implementazione Parallela con CUDA.....	21
4.1 Architettura Ibrida CPU-GPU.....	21
4.2 Riduzione Parallela su GPU	22
4.3 Implementazione del Kernel	24
4.4 Flusso di Esecuzione	25
CAPITOLO 5: Test e Analisi	26
5.1 Metodologia Di Generazione Del Dataset Di Test.....	26
5.1.1 xorshift64*	26
5.1.2 Analisi e Gestione dell'Overhead	28
5.2 Experimental Setup	30
5.3 Analisi Comparativa delle Prestazioni.....	31
5.3.1 Impatto delle Ottimizzazioni del Compilatore.....	31
5.3.2 Breakdown Temporale per Fase	32

5.3.3 Confronto Diretto: MPI vs CUDA	33
5.3.4 Effetto del Block Size su CUDA	34
5.3.5 Scalabilità MPI: Efficienza Parallela	36
5.3.6 Speedup Aggregato per Tutte le Ottimizzazioni	37
5.3.7 Comportamento su Diverse Dimensioni di Input	38
5.3.8 Indipendenza dell'Efficienza dalle Ottimizzazioni	39
5.4 Discussione dei Risultati e Conclusioni	39
Bibliografia	41

INTRODUZIONE

Il presente lavoro documenta la progettazione, l'implementazione e l'analisi comparativa di soluzioni parallele dell'algoritmo di Manber-Myers, finalizzate alla costruzione di Suffix Array e alla conseguente identificazione della *Longest Repeated Substring* (LRS). Sebbene questa struttura dati offra una soluzione compatta ed efficiente, la costruzione su stringhe di grandi dimensioni pone sfide computazionali che spesso eccedono le capacità di una singola CPU.

L'obiettivo primario del progetto è affrontare tali limitazioni valutando come due distinte strategie di parallelizzazione, una basata su **MPI** (*Message Passing Interface*) per sistemi a memoria distribuita e una su **CUDA** per l'accelerazione tramite GPU, si comportino rispetto a un'implementazione sequenziale di riferimento. Il confronto verterà sia sulla correttezza dei risultati che sull'efficienza prestazionale, verificate attraverso un approccio sperimentale su dataset di dimensioni incrementali.

Il lavoro è strutturato per rispondere ai seguenti obiettivi operativi:

- 1. Definizione della Baseline Sequenziale** Selezione e adattamento di un'implementazione sequenziale dell'algoritmo di Manber-Myers in linguaggio C. Questa versione fungerà da riferimento per la misurazione delle prestazioni e per la verifica dell'output delle versioni parallele.
- 2. Parallelizzazione su Memoria Distribuita (MPI)** Realizzazione di una versione parallela basata sullo standard *Message Passing Interface* (MPI). L'obiettivo è distribuire il carico di lavoro su più nodi di calcolo, permettendo la gestione di stringhe che eccedono la memoria di una singola macchina e riducendo i tempi di esecuzione attraverso la cooperazione tra processi distinti.
- 3. Parallelizzazione su Architettura Many-Core (CUDA)** Sviluppo di una versione accelerata mediante tecnologia NVIDIA CUDA. Questa implementazione mira a sfruttare le risorse della GPU per gestire in modo efficiente le operazioni parallelizzabili dell'algoritmo, con l'obiettivo di valutare i benefici prestazionali ottenibili rispetto alla controparte sequenziale.
- 4. Metodologia di Test e Valutazione Prestazionale** Esecuzione di test su un set di stringhe generate casualmente con dimensioni prefissate (1MB, 50MB, 100MB, 200MB, 500MB). Al fine di garantire la riproducibilità e l'equità del confronto, i dataset verranno generati, salvati su file e forniti in input identici a tutte e tre le implementazioni. L'analisi finale confronterà tempi di esecuzione, *speedup* ed efficienza, evidenziando i vantaggi e i limiti di ciascuna architettura al variare della dimensione del problema.

CAPITOLO 1: Fondamenti

Il presente capitolo illustra i fondamenti teorici necessari alla comprensione del progetto, definendo formalmente il problema algoritmico e le strutture dati adottate. In particolare, verrà mostrata la struttura **Suffix Array** e verrà descritto in dettaglio l'algoritmo di **Manber-Myers**. Infine, si introdurrà il problema della ricerca della *Longest Repeated Substring (LRS)*, la cui risoluzione costituisce l'obiettivo finale dell'implementazione proposta.

1.1 Suffix Array

Il **Suffix Array** (SA) è una struttura dati progettata per facilitare operazioni complesse come la ricerca di sottostringhe e l'analisi di pattern ripetuti. Formalmente, dato un testo S di lunghezza n , il Suffix Array è definito come un array di interi che rappresenta una permutazione degli indici da 0 a $n - 1$. Ogni intero $SA[i]$ rappresenta l'indice di partenza di un suffisso del testo S .

La proprietà fondamentale che governa questa struttura risiede nell'ordinamento lessicografico: gli indici in SA sono disposti in modo tale che i suffissi corrispondenti appaiano in ordine crescente. Definito S_j come il suffisso che inizia alla posizione j , ovvero la sottostringa $S[j \dots n - 1]$, il Suffix Array deve soddisfare la seguente relazione d'ordine:

$$S_{SA[0]} <_{lex} S_{SA[1]} <_{lex} \dots <_{lex} S_{SA[n-1]}$$

Questa organizzazione trasforma il problema della ricerca di una stringa W di lunghezza P all'interno di un testo di lunghezza N in un problema di ricerca binaria. Poiché i suffissi sono ordinati, tutte le occorrenze di W appariranno in posizioni contigue all'interno del Suffix Array. Di conseguenza, è possibile determinare la presenza e la posizione di W con una complessità temporale di $O(P \log N)$ nel caso peggiore, effettuando confronti diretti tra il pattern e i suffissi indicizzati dall'array,

Esempio:

Consideriamo la stringa $S = \text{"banana\$"}$

Il carattere speciale $\$$ funge da terminatore univoco e, per convenzione, è considerato lessicograficamente minore di qualsiasi altro carattere dell'alfabeto.

La lunghezza totale della stringa è $n = 7$ (indici $0 \dots 6$).

Il primo passo consiste nell'identificare tutti i suffissi generati dalla stringa associandoli ai loro indici di partenza originali:

- **Indice 0:** banana\$
- **Indice 1:** anana\$
- **Indice 2:** nana\$
- **Indice 3:** ana\$
- **Indice 4:** na\$
- **Indice 5:** a\$
- **Indice 6:** \$

Applicando l'ordinamento lessicografico a tale elenco, si ottiene la seguente sequenza ordinata:

1. \$ (corrispondente all'indice **6**)
2. a\$ (corrispondente all'indice **5**)
3. ana\$ (corrispondente all'indice **3**)
4. anana\$ (corrispondente all'indice **1**)
5. banana\$ (corrispondente all'indice **0**)
6. na\$ (corrispondente all'indice **4**)
7. nana\$ (corrispondente all'indice **2**)

Il Suffix Array risultante è quindi il vettore di interi che riflette questo ordinamento posizionale: **SA = [6, 5, 3, 1, 0, 4, 2]**

Come si evince dall'esempio, SA[0]=6 punta al suffisso lessicograficamente più piccolo (\$), SA[1]=5 al secondo (a\$), e così via. Questa struttura composta esclusivamente da n interi, racchiude l'informazione completa sull'ordinamento del testo.

1.2 Algoritmo di Manber-Myers

Per la costruzione del Suffix Array, il progetto adotta l'algoritmo introdotto da Udi Manber e Gene Myers nel 1990. Questo approccio rappresenta una soluzione efficiente per l'ordinamento dei suffissi, garantendo una complessità temporale di $O(n \log n)$ nel caso peggiore, dove n è la lunghezza della stringa di input.

L'efficienza di questo algoritmo si deve all'adozione della tecnica incrementale nota come **Prefix Doubling**. L'intuizione fondamentale è che, per ordinare due suffissi in base ai loro primi $2h$ caratteri, non è necessario confrontare i caratteri uno per uno se si conosce già l'ordinamento relativo dei loro prefissi di lunghezza h . Sfruttando questa proprietà induttiva, l'algoritmo procede per stadi logaritmici: all'iterazione k , i suffissi vengono ordinati considerando prefissi di lunghezza $h = 2^k$. In questo modo, la lunghezza del prefisso ordinato raddoppia ad ogni passo (1, 2, 4, 8, ...), permettendo di completare l'ordinamento in $O(\log n)$ iterazioni totali.

Inoltre, per rendere efficiente il confronto tra prefissi di lunghezza crescente senza accedere continuamente al testo originale, l'algoritmo introduce il concetto di **rank**. Un rango è un numero intero che rappresenta la posizione lessicografica del prefisso parziale del suffisso nell'ordinamento calcolato allo stadio precedente. Per ordinare i suffissi allo stadio $2h$, a ciascun suffisso S_i viene associata una **tupla di ordinamento** composta da due chiavi intere dove la prima è il rango del prefisso di lunghezza h che inizia alla posizione i , mentre la seconda è il rango del prefisso di lunghezza h che inizia alla posizione $i + h$.

La potenza risiede nel fatto che la seconda chiave "guarda avanti" di h posizioni. Matematicamente, se due suffissi hanno la stessa prima chiave, l'ordinamento viene deciso dalla seconda chiave. Poiché entrambi i valori sono interi calcolati nell'iterazione precedente, il confronto avviene in tempo costante $O(1)$, indipendentemente dalla lunghezza h del prefisso considerato.

1.3 Algoritmo di Kasai

Una volta costruito il Suffix Array e ottenuto l'ordinamento lessicografico dei suffissi, diventa necessario disporre di informazioni sulla similarità tra i suffissi adiacenti per supportare analisi testuali più avanzate. A questo scopo interviene l'**Array LCP** (*Longest Common Prefix*), che registra per ciascun suffisso la lunghezza del prefisso condiviso con il suffisso che lo precede nell'ordinamento.

Sebbene l'algoritmo originale di Manber-Myers preveda la possibilità di calcolare il Longest Common Prefix come sottoprodotto della costruzione del Suffix Array, nella pratica si tende a separare i due problemi e calcolare l'LCP utilizzando l'**algoritmo di Kasai et al.** (2001) che offre una soluzione in tempo lineare $O(n)$.

La strategia di questo algoritmo consiste nel cambiare l'ordine di elaborazione, infatti, invece di calcolare i valori LCP seguendo l'ordine del Suffix Array che è casuale rispetto alla posizione nel testo, l'algoritmo elabora i suffissi seguendo il loro ordine di apparizione nel testo originale. Questo approccio permette di sfruttare le forti correlazioni esistenti tra suffissi consecutivi, evitando confronti ridondanti.

L'efficienza dell'algoritmo si regge su un teorema fondamentale, spesso citato come **Lemma di Kasai**. Tale proprietà stabilisce una relazione matematica tra l'LCP di un suffisso e l'LCP del suffisso che inizia alla posizione immediatamente successiva. Formalmente, se si considera un suffisso S_i e il suo predecessore lessicografico e si denota con h la lunghezza del loro prefisso comune, il lemma afferma che il suffisso successivo S_{i+1} condividerà con il suo rispettivo predecessore lessicografico un prefisso di lunghezza almeno $h - 1$.

L'intuizione alla base è che se S_i condivide un prefisso lungo h con un altro suffisso S_j , rimuovendo il primo carattere da entrambi si ottengono i suffissi S_{i+1} e S_{j+1} , i quali rimarranno identici per i restanti $h - 1$ caratteri.

Questa proprietà teorica consente di sapere che il match del suffisso successivo sarà quasi lungo quanto quello attuale meno un carattere per cui non è necessario riavviare i confronti da zero ma è sufficiente riprendere il confronto a partire dal numero di caratteri già garantiti dal lemma.

1.4 Longest Repeated Substring

L'identificazione di pattern ricorrenti è un'operazione essenziale per comprendere la struttura interna di un dato testuale. Tra le diverse metriche di regolarità, il problema della *Longest Repeated Substring* (**LRS**) mira a quantificare la massima ridondanza contigua presente in una sequenza, isolando la porzione di testo più lunga che si ripete identica a sé stessa. In termini formali, data una stringa S su un alfabeto finito, il problema consiste nell'individuare una sottostringa ω tale che appaia in S con una frequenza maggiore o pari a 2 e la cui lunghezza ω sia massimizzata.

Se si utilizzasse un approccio brute-force dove si confronta ogni possibile sottostringa con tutte le altre, il problema presenterà una complessità computazionale proibitiva. La sfida risiede dunque nell'abbattere questa barriera, trasformando la ricerca in un processo trattabile linearmente o quasi-linearmente rispetto alla lunghezza del testo.

La soluzione al problema LRS si basa su una trasformazione strutturale dei dati. L'intuizione chiave è che ogni sottostringa ripetuta nel testo è, per definizione, il prefisso di almeno due suffissi distinti della stringa originale. Il **Suffix Array** raggruppa spazialmente queste occorrenze e, grazie alla proprietà di ordinamento lessicografico, si ha che tutti i suffissi che iniziano con la medesima sequenza di caratteri vengono posizionati in locazioni contigue all'interno dell'array. Ciò trasforma un problema di ricerca globale in un problema di confronto locale poiché le occorrenze più simili di una qualsiasi sottostringa si troveranno necessariamente vicine nel Suffix Array. L'Array LCP, integra questa informazione con la lunghezza esatta dei prefissi condivisi tra suffissi adiacenti e quantifica numericamente questa similarità.

CAPITOLO 2: Implementazione Sequenziale

Questa implementazione costituisce la baseline fondamentale del progetto, necessaria per validare la correttezza delle future versioni parallele e per stabilire i riferimenti prestazionali. L'architettura software è sviluppata in C standard (C99). Il flusso di esecuzione segue una pipeline modulare in tre stadi: costruzione del Suffix Array, costruzione dell'LCP Array e la ricerca dell'LRS.

Il progetto si articola su tre file principali:

- **seq_suffix_array.h**: all'interno di questo file header sono definite le strutture dati fondamentali, tra cui la struct **Suffix** utilizzata per l'ordinamento, e i prototipi delle funzioni pubbliche **suffixSort()**, **getHeight()** e **findLRS()**.
- **seq_suffix_array.c**: costituisce il nucleo computazionale della versione sequenziale. Questo file contiene l'implementazione concreta dell'algoritmo di **Manber-Myers** per la costruzione del Suffix Array e dell'algoritmo di **Kasai** per il calcolo dell'array LCP.
- **seq_main.c**: è il punto di ingresso del programma. Questo modulo ha la responsabilità di gestire il flusso di esecuzione ad alto livello, infatti, si occupa del caricamento del dataset binario in memoria, dell'allocazione delle risorse, dell'invocazione sequenziale degli algoritmi di costruzione e, infine, della misurazione dei tempi di esecuzione e della presentazione dei risultati.
- **common_functions.c**: è un modulo di supporto che raccoglie le utilità trasversali condivise con le implementazioni parallele, limitandosi essenzialmente alla gestione dell'I/O e alle routine di misurazione del tempo.

2.1 Architettura e Scelte Implementative

Una decisione implementativa importante riguarda la rappresentazione interna della stringa di input. Sebbene i file di input siano sequenze di byte binari, il sistema converte immediatamente questi dati in un array di interi a 32 bit (**int *text**) durante la fase di lettura. Questa scelta, apparentemente dispendiosa in termini di memoria poiché da 1 byte per carattere si passa a 4 byte per carattere, consente di gestire alfabeti con più simboli e una maggiore compatibilità con i **rank** gestiti dall'algoritmo di Manber-Myers.

Inoltre, per facilitare le operazioni di ordinamento, è stata definita una struttura dati dedicata:

```
typedef struct {  
    int index;  
    int rank[2];  
} Suffix;
```

Figura 1: Codice della struttura Suffix

- Il campo **index** contiene l'indice di partenza del suffisso S_i all'interno del testo. Al termine del processo di ordinamento, la sequenza di questi indici costituirà il Suffix Array finale **sa**.
- Il campo **rank[0]** (*Rank Corrente*) rappresenta il rango lessicografico del prefisso di lunghezza **h** che inizia alla posizione **index**. Questo valore viene ereditato dall'iterazione precedente dell'algoritmo, ovvero dopo l'iterazione **k**.

- Il campo **rank[1]** (*Rank Successivo*) rappresenta il rango lessicografico del prefisso di lunghezza **h** che inizia alla posizione **index + h**. Questo permette di ordinare i blocchi di lunghezza **2h** confrontando la coppia (**rank[0]**, **rank[1]**).

Se $\text{index} + h \geq n$ viene assegnato un valore sentinella -1 che, per convenzione, è minore di qualsiasi rango valido, garantendo che l'ordine sia corretto. In termini formali, la sentinella agisce come un carattere fittizio, garantendo la corretta terminazione dei confronti lessicografici.

2.2 Implementazione dell'Algoritmo di Manber-Myers

La funzione **suffixSort()** implementa la logica del **Prefix Doubling**. L'algoritmo non ordina i suffissi in un singolo passaggio, ma procede per un numero logaritmico di iterazioni. In ogni fase **k**, i suffissi vengono ordinati in base a prefissi di lunghezza $h = 2^k$, sfruttando l'ordinamento calcolato nella fase precedente.

Prima di descrivere l'implementazione della funzione, è importanti sottolineare l'utilizzo di strutture di supporto quali l'array **suffixes[]** che contiene **n** strutture **suffix**, una per ogni suffisso del testo mentre **ind2rank[]** è l'array inverso che, dato l'indice di partenza **i** di un suffisso, restituisce la sua posizione attuale nell'array **suffixes** ordinato.

```
Suffix *suffixes = (Suffix *)malloc(n * sizeof(Suffix));
int *ind2rank = (int *)malloc(n * sizeof(int));
```

Figura 2: Codice strutture di supporto

L'algoritmo di Manber-Myers si articola in diverse fasi:

1. **Prima iterazione:** Inizializzazione $h = 1$

Questa fase preliminare ha l'obiettivo di preparare tutte le strutture dati per ordinare i suffissi in base ai primi due caratteri. L'inizializzazione di **suffixes[i].index = i** è importante perché a questo punto l'ordine nell'array **suffixes** coincide ancora con l'ordine posizionale nel testo mentre successivamente non sarà più così. In questo modo si potrà risalire alla posizione originale di partenza del suffisso. Come prima chiave di ordinamento viene utilizzato **text[i]** mentre la seconda chiave viene inizializzata con il carattere successivo **text[i+1]**.

```
for (int i = 0; i < n; i++) {
    suffixes[i].index = i;
    suffixes[i].rank[0] = text[i];
    suffixes[i].rank[1] = ((i + 1) < n) ? text[i + 1] : -1;
}
```

Figura 3: Codice fase 1 Algoritmo di Manber-Myers

2. Primo ordinamento:

Una volta inizializzate le chiavi di ordinamento per $h = 1$, viene invocata la funzione standard `qsort` per ordinare l'array `suffixes`. La logica di ordinamento è incapsulata nella funzione di callback `compare_suffixes`, la quale implementa un confronto lessicografico gerarchico operando sui puntatori generici `void *`:

```
int compare_suffixes(const void *a, const void *b) {
    const Suffix *s1 = (const Suffix *)a;
    const Suffix *s2 = (const Suffix *)b;
    int diff = s1->rank[0] - s2->rank[0];
    return diff ? diff : (s1->rank[1] - s2->rank[1]);
}
```

Figura 4: Codice fase 2 Algoritmo di Manber-Myers

La funzione stabilisce l'ordine tra due suffissi S_1 e S_2 valutando le chiavi in due stadi successivi. Inizialmente si confronta la prima chiave e, se i valori sono differenti, l'ordine viene già stabilito, altrimenti si ricorre alla seconda chiave. Ciò garantisce che anche i suffissi aventi lo stesso carattere iniziale vengano ordinati correttamente.

3. Prefix Doubling:

Questo è il ciclo iterativo che estende progressivamente l'ordinamento dei suffissi. Si parte dall'ordinamento basato sui primi $h = 1$ caratteri e il ciclo raddoppia la lunghezza del prefisso considerato ad ogni passo fino a quando $h < n$.

- I. **Assegnazione dei nuovi rank e aggiornamento mappa inversa:** dopo un ordinamento, è necessario riassegnare i ranghi lessicografici in modo che i suffissi che risultano identici per i primi h caratteri condividano lo stesso rank, mentre suffissi distinti abbiano valori di rank crescenti. Contestualmente, si aggiorna l'array `ind2rank`. Questa scansione lineare comprime l'informazione di ordinamento della coppia $(rank[0], rank[1])$ in un singolo intero `rank`.

Terminazione Anticipata: l'istruzione `if (rank == n - 1) break;` verifica se tutti i suffissi hanno ricevuto un rank distinto. In tal caso, l'ordinamento lessicografico è completo e univoco, rendendo inutili ulteriori iterazioni.

- II. **Aggiornamento coppie per la prossima iterazione $2h$:** per preparare l'ordinamento sulla lunghezza $2h$, si deve costruire la nuova chiave secondaria `rank[1]`. Per un suffisso che inizia in posizione i , la seconda metà del prefisso di lunghezza $2h$ corrisponde al prefisso di lunghezza h che inizia in posizione $i + h$. La logica sfrutta i rank appena calcolati, ovvero l'ordine sui primi h caratteri, in modo da calcolare `next_index = index + h` e usare `ind2rank[next_index]` per trovare dove si trova quel suffisso nell'array ordinato. Si procede poi con il prelevare il suo `rank[0]` e lo si assegna come `rank[1]` del suffisso corrente. Nel caso in cui `next_index` eccede la lunghezza del testo, si assegna la sentinella -1.

- III. **Riordinamento:** infine, l'array viene riordinato in base alle nuove coppie aggiornate. Poiché `rank[0]` riassume ora l'ordine dei primi h caratteri e

rank[1] l'ordine della seconda metà, confrontare le coppie equivale a confrontare i suffissi sulla lunghezza combinata $2h$.

- IV. Dopo l'ultima iterazione del ciclo, `suffixes[]` contiene i suffissi in ordine lessicografico completo. Si estraggono quindi i campi `index` per popolare l'array di output `sa[]`.

```
for (int h = 2; h < 2 * n; h *= 2) {
    int rank = 0;
    int prev_rank = suffixes[0].rank[0];
    suffixes[0].rank[0] = rank;
    ind2rank[suffixes[0].index] = 0;

    for (int i = 1; i < n; i++) {
        if (suffixes[i].rank[0] == prev_rank &&
            suffixes[i].rank[1] == suffixes[i-1].rank[1]) {
            prev_rank = suffixes[i].rank[0];
            suffixes[i].rank[0] = rank;
        } else {
            prev_rank = suffixes[i].rank[0];
            suffixes[i].rank[0] = ++rank;
        }
        ind2rank[suffixes[i].index] = i;
    }

    if (rank == n - 1) break;

    for (int i = 0; i < n; i++) {
        int next_index = suffixes[i].index + h;
        suffixes[i].rank[1] = (next_index < n) ?
                               suffixes[ind2rank[next_index]].rank[0] : -1;
    }

    qsort(suffixes, n, sizeof(Suffix), compare_suffixes);
}
```

Figura 5: Codice fase 3 Algoritmo di Manber-Myers

2.3 Implementazione dell'Algoritmo di Kasai

Per il calcolo del *Longest Common Prefix (LCP)*, l'implementazione si affida all'algoritmo lineare proposto da Kasai et al. (2001), implementato nella funzione `getHeight()`.

Sebbene Manber e Myers (1993) avessero originariamente proposto di calcolare i valori LCP contestualmente alla costruzione del Suffix Array sfruttando i risultati intermedi dell'ordinamento, tale approccio presenta limitazioni ingegneristiche significative. Esso tende a mescolare la logica di ordinamento con quella di analisi e mantiene una complessità asintotica dominata dal fattore $O(n \log n)$. Al contrario, l'algoritmo di Kasai offre un approccio che garantisce una complessità lineare $O(n)$, risultando asintoticamente più efficiente.

L'algoritmo di Kasai adotta una strategia di visita basata sull'ordine testuale dei suffissi $i = 0, \dots, n - 1$ e non sul loro ordine lessicografico. Tuttavia, per calcolare il valore LCP, è necessario confrontare il suffisso corrente i con il suo **predecessore lessicografico**. Per rendere efficiente questa operazione, è necessario costruire preliminarmente un array di supporto, denominato `rank`, che rappresenta la permutazione inversa del Suffix Array. Formalmente, se $SA[k] = i$, allora deve valere $rank[i] = k$.

La necessità di questa struttura è dettata dall'efficienza, infatti, senza l'array `rank`, individuare la posizione lessicografica di un suffisso i richiederebbe una scansione lineare di SA , degradando la complessità totale. Grazie a `rank` invece l'operazione diventa $O(1)$.

Il cuore dell'algoritmo risiede nell'applicazione del **Lemma di Kasai**, che stabilisce una proprietà di non-decremento della lunghezza del prefisso comune. Infatti, il lemma afferma che se il suffisso che inizia alla posizione i condivide un prefisso di lunghezza h con il suo predecessore lessicografico, allora il suffisso successivo $i + 1$ condividerà con il suo rispettivo predecessore un prefisso di lunghezza almeno $h - 1$.

Questa proprietà permette di evitare il reset del contatore h a zero ad ogni iterazione. Il confronto per il suffisso $i + 1$ può iniziare direttamente dal carattere in posizione $h - 1$, evitando confronti ridondanti su caratteri già noti.

Di seguito l'implementazione della logica:

```
void getHeight(int *text, int *sa, int *lcp, int n) {
    int *rank = (int *)malloc(n * sizeof(int));
    if (!rank) {
        fprintf(stderr, "ERRORE: Impossibile allocare memoria per rank in getHeight\n");
        return;
    }

    for (int i = 0; i < n; i++) {
        rank[sa[i]] = i;
    }

    int h = 0;
    lcp[0] = 0;

    for (int i = 0; i < n; i++) {
        if (rank[i] > 0) {
            int j = sa[rank[i] - 1];
            while (i + h < n && j + h < n && text[i + h] == text[j + h]) {
                h++;
            }
            lcp[rank[i]] = h;
            if (h > 0) h--;
        }
    }

    free(rank);
}
```

Figura 6: Codice della funzione `getHeight`

2.4 Ricerca della Longest Repeated Substring

La fase conclusiva della pipeline di elaborazione consiste nell'identificazione della Longest Repeated Substring. Grazie alla preparazione dei dati effettuata nelle fasi precedenti, il problema si riduce a una semplice operazione di scansione lineare. La funzione `findLRS()` implementa questa logica:

```
int findLRS(int *lcp, int n) {
    int maxLCP = 0;
    for (int i = 1; i < n; i++) {
        if (lcp[i] > maxLCP) {
            maxLCP = lcp[i];
        }
    }
    return maxLCP;
}
```

Figura 7: Codice della funzione `findLRS`

La correttezza di questo approccio si basa su un teorema fondamentale dell'algorithmica su stringhe: **la lunghezza della sottostringa ripetuta più lunga in un testo T corrisponde esattamente al valore massimo contenuto nell'array LCP di T** . La dimostrazione intuitiva di tale proprietà poggia su due concetti chiave:

- **Adiacenza nel Suffix Array:** se una sottostringa compare più volte nel testo, essa costituisce il prefisso di molteplici suffissi. Poiché il Suffix Array è ordinato lessicograficamente, tutti i suffissi che condividono un determinato prefisso vengono raggruppati in posizioni contigue all'interno dell'array.
- **Transitività Lessicografica:** per la proprietà transitiva dell'ordinamento, se due suffissi distanti S_i e S_j condividono un prefisso di lunghezza k , allora tutti i suffissi compresi tra i e j devono necessariamente condividere un prefisso di lunghezza maggiore o pari a k . Di conseguenza, esisterà almeno una coppia di suffissi immediatamente adiacenti che condivide quel prefisso.

Pertanto, non è necessario confrontare tutte le coppie possibili ma è sufficiente esaminare i valori LCP delle sole coppie adiacenti per individuare il massimo globale.

L'algoritmo esegue una singola iterazione sull'array `lcp`, partendo dall'indice $i = 0$ visto che `LCP[0]` è per definizione nullo, non avendo un predecessore. Tale operazione richiede un singolo ciclo su n elementi, con operazioni elementari di confronto e assegnazione a costo costante $O(1)$. La complessità è quindi strettamente lineare $O(n)$. Una volta ottenuto `maxIndex`, la sottostringa può essere ricostruita accedendo al testo originale tramite l'indice salvato nel Suffix Array.

2.5 Flusso di Esecuzione

Il flusso logico dell'implementazione sequenziale può essere schematizzato nei seguenti passaggi operativi:

1. **Caricamento e Preprocessing:** lettura del dataset binario e conversione in memoria nell'array di interi `text`.
2. **Costruzione Suffix Array:** ordinamento dei suffissi e popolamento dell'array `sa` mediante l'algoritmo di Manber-Myers.
3. **Calcolo LCP:** costruzione dell'array `lcp` utilizzando l'algoritmo lineare di Kasai.
4. **Estrazione LRS:** identificazione della Longest Repeated Substring determinando il valore massimo all'interno dell'array LCP: $LRS_{len} = \max_k lcp[k]$

CAPITOLO 3: Implementazione Parallela MPI

L'implementazione MPI nasce dall'esigenza di superare i limiti di memoria di un singolo processore, distribuendo il carico di lavoro su più nodi. La sfida principale di questo approccio risiede nella natura intrinsecamente globale dell'ordinamento: confrontare due suffissi può richiedere l'accesso a porzioni di testo che risiedono su memorie fisicamente distanti, impedendo di fatto un'elaborazione puramente locale e indipendente. Per risolvere tale vincolo, la strategia adottata combina la decomposizione del dominio con un meccanismo di unione gerarchica. L'architettura risultante si articola in tre fasi sequenziali: un'elaborazione locale distribuita, una fase di raccolta e fusione centralizzata e un post-processing finale.

Il progetto si articola su tre file principali:

- **mpi_main.c**: gestisce l'inizializzazione dell'ambiente MPI, la sincronizzazione tra processi e la misurazione dei tempi di esecuzione. Coordina l'invocazione delle funzioni di lettura, ordinamento locale e merge globale.
- **mpi_suffix_array.c**: contiene l'implementazione delle due funzioni principali, `read_and_sort_chunk()` che gestisce l'I/O parallelo e la costruzione locale dei Suffix Array e `gather_and_merge()` che raccoglie i risultati parziali sul processo coordinatore Rank 0 e li fonde in un Suffix Array globale.
- **merge_algorithms.c**: implementa la logica di merge iterativo tramite `iterative_merge()` che combina logaritmicamente i Suffix Array parziali e la funzione di merge a due vie `merge_two_suffix_arrays()` che costituisce il nucleo del confronto lessicografico tra suffissi distribuiti.

3.1 Elaborazione Distribuita e Ordinamento Locale

La prima fase della pipeline, implementata nella funzione **read_and_sort_chunk**, affronta il problema dell'acquisizione efficiente dei dati. Per garantire un buon bilanciamento del carico, il file di input non viene letto da un singolo processo master, ma partizionato logicamente tra tutti i P processi disponibili.

La dimensione del blocco assegnato a ciascun processo di rank r è calcolata dinamicamente per gestire il caso in cui la dimensione totale del file N non sia perfettamente divisibile per il numero di processi P . Una divisione intera genererebbe un resto $R = N \bmod P$ byte senza assegnazione. Invece di assegnare tale resto in blocco all'ultimo processo e creare un potenziale squilibrio di carico, l'algoritmo distribuisce questi byte in eccesso in modalità *round-robin* ai primi R processi, ovvero quelli con $rank < R$.

La dimensione locale è quindi formalizzata come:

$$size_r = N/P + (r < (N \bmod P) ? 1 : 0)$$

In questa equazione, N/P rappresenta il numero minimo di byte garantiti a ogni processo mentre il termine condizionale aggiunge un singolo byte di "padding" solo ai primi $N \bmod P$ processi.

Questa strategia garantisce matematicamente che il carico di lavoro differisca al massimo di un singolo byte tra due processi, realizzando un buon bilanciamento del carico e minimizzando i tempi di attesa nelle barriere di sincronizzazione successive.

Una volta determinata la dimensione, l'offset di lettura nel file globale, ovvero il puntatore di inizio per ogni processo, viene derivato sommando le dimensioni di tutti i chunk precedenti:

$$offset_r = r \times N/P + \min(r, N \bmod P)$$

Il termine $\min(r, N \bmod P)$ calcola quanti byte in più sono stati distribuiti ai processi con rank inferiore a quello corrente, garantendo che ogni processo inizi la lettura esattamente dove il processo $r - 1$ ha terminato, senza sovrapposizioni né perdita di dati.

```
void read_and_sort_chunk(const char *filename, int rank, int size, int **local_text, int **local_sa, long *out_chunk_size, long *out_total_size, MPI_Offset *out_offset){
    MPI_File fh;
    int err = MPI_File_open(MPI_COMM_WORLD, filename, MPI_MODE_RDONLY, MPI_INFO_NULL, &fh);

    if (err != MPI_SUCCESS) {
        if (rank == 0) {
            fprintf(stderr, "ERRORE CRITICO: Impossibile aprire il file '%s'\n", filename);
        }
        MPI_Abort(MPI_COMM_WORLD, 1);
    }

    MPI_Offset total_len;
    MPI_File_get_size(fh, &total_len);
    *out_total_size = (long)total_len;

    long base_chunk = *out_total_size / size;
    long remainder = *out_total_size % size;

    *out_chunk_size = base_chunk + (rank < remainder ? 1 : 0);
    *out_offset = rank * base_chunk + (rank < remainder ? rank : remainder);

    unsigned char *buffer = (unsigned char*)malloc(*out_chunk_size);
    if (!buffer) {
        fprintf(stderr, "[Rank %d] Errore allocazione buffer\n", rank);
        MPI_Abort(MPI_COMM_WORLD, 1);
    }
}
```

Figura 8: Codice della funzione `read_and_sort_chunk`

L'accesso al disco è gestito tramite il sottosistema **MPI-IO**. Utilizzando la primitiva collettiva **MPI_File_read_at_all**, tutti i processi accedono simultaneamente al file system parallelo, leggendo la propria porzione in modo indipendente. Questo approccio elimina il collo di bottiglia tipico della lettura serializzata e permette alla banda di I/O di scalare con il numero di nodi. Immediatamente dopo la lettura, i byte vengono convertiti in un array di interi (`int*`) per garantire la compatibilità con le strutture dati dell'algoritmo di Manber-Myers.

```
MPI_File_read_at_all(fh, *out_offset, buffer, (int)(*out_chunk_size), MPI_BYTE, MPI_STATUS_IGNORE);
MPI_File_close(&fh);
```

Figura 9: Estratto della funzione `read_and_sort_chunk`

Una volta che il segmento di testo è disponibile in memoria locale, ciascun processo procede autonomamente all'ordinamento dei suffissi invocando la funzione sequenziale **suffixSort** sul proprio *chunk*.

È fondamentale osservare che il Suffix Array risultante da questa operazione contiene indici **locali**, validi esclusivamente all'interno del frammento assegnato. Tuttavia, la successiva fase di integrazione richiede un sistema di riferimento univoco per l'intero dataset. Pertanto, prima di avviare qualsiasi comunicazione verso il processo Master, viene eseguita un'operazione di **globalizzazione degli indici**: a ogni elemento del Suffix Array locale viene sommato l'offset di

lettura calcolato inizialmente tramite $local_sa[i] += offset_r$. Questa trasformazione mappa lo spazio degli indirizzi locali nello spazio globale, garantendo che ogni suffisso punti correttamente alla sua posizione assoluta nel file originale. Senza quest'operazione, il processo di merge non avrebbe modo di ricostruire l'ordine lessicografico corretto tra suffissi appartenenti a nodi diversi.

```
*local_text = (int*)malloc((*out_chunk_size) * sizeof(int));
if (!*local_text) {
    fprintf(stderr, "[Rank %d] Errore allocazione local_text\n", rank);
    free(buffer);
    MPI_Abort(MPI_COMM_WORLD, 1);
}

for (long i = 0; i < *out_chunk_size; i++) {
    (*local_text)[i] = (int)buffer[i];
}
free(buffer);

*local_sa = (int*)malloc((*out_chunk_size) * sizeof(int));
if (!*local_sa) {
    fprintf(stderr, "[Rank %d] Errore allocazione local_sa\n", rank);
    free(*local_text);
    MPI_Abort(MPI_COMM_WORLD, 1);
}

suffixSort(*local_text, *local_sa, (int)(*out_chunk_size));

for (long i = 0; i < *out_chunk_size; i++) {
    (*local_sa)[i] += (int)(*out_offset);
}
```

Figura 10: Estratto della funzione `read_and_sort_chunk`

3.2 Raccolta Dati e Preparazione al Merge

Terminata l'elaborazione locale, l'architettura prevede una fase di **centralizzazione** dei risultati sul nodo coordinatore Rank 0. Questa operazione, guidata dalla funzione **gather_and_merge**, è propedeutica alla fusione finale e deve soddisfare due necessità strutturali distinte. La prima è la disponibilità del testo completo per i confronti e la seconda consiste nell'acquisizione delle strutture ordinate parziali.

1. Ricostruzione del Testo Globale

Poiché il confronto tra suffissi situati su nodi diversi richiede di poter leggere caratteri in qualsiasi punto della stringa, è indispensabile che il Rank 0 disponga dell'intero testo in memoria. Per la ricostruzione si utilizza la primitiva **MPI_Gatherv**. A differenza di una Gather standard che richiede dati di dimensione fissa, questa funzione è progettata per gestire chunk di dimensioni variabili, una condizione inevitabile dovuta al bilanciamento del carico nella fase precedente. Grazie ai vettori di supporto `recvcounts` (per le dimensioni) e `displs` (per gli

offset), **MPI_Gatherv** riassembla correttamente i vari frammenti irregolari in un unico array contiguo `global_text`.

```
void gather_and_merge(int *local_text, int *local_sa, long chunk_size, int rank, int size, long total_n, int **global_text, int **global_sa) {
    int *recvcnts = NULL;
    int *displs = NULL;

    if (rank == 0) {
        *global_text = (int*)malloc(total_n * sizeof(int));
        *global_sa = (int*)malloc(total_n * sizeof(int));
        recvcnts = (int*)malloc(size * sizeof(int));
        displs = (int*)malloc(size * sizeof(int));

        if (!*global_text || !*global_sa || !recvcnts || !displs) {
            fprintf(stderr, "[Rank 0] ERRORE: Allocazione memoria per gather\n");
            MPI_Abort(MPI_COMM_WORLD, 1);
        }
    }

    int my_size = (int)chunk_size;
    MPI_Gather(&my_size, 1, MPI_INT, recvcnts, 1, MPI_INT, 0, MPI_COMM_WORLD);

    if (rank == 0) {
        displs[0] = 0;
        for (int i = 1; i < size; i++) {
            displs[i] = displs[i-1] + recvcnts[i-1];
        }
    }

    MPI_Gatherv(local_text, my_size, MPI_INT, *global_text, recvcnts, displs, MPI_INT, 0, MPI_COMM_WORLD);
}
```

Figura 11: Estratto della funzione `gather_and_merge`

2. Aggregazione dei Suffix Array Parziali

Parallelamente al testo, il nodo master deve acquisire i Suffix Array calcolati dai singoli worker. In questo caso, l'obiettivo non è la concatenazione cieca, bensì la conservazione della struttura ordinata di ciascun blocco. Per questo motivo, invece di una collettiva, si adotta uno schema di comunicazione punto-a-punto (**MPI_Send/MPI_Recv**).

Parallelamente al testo, il nodo Master deve raccogliere i Suffix Array calcolati dai singoli processi. A differenza del testo, però, questi array non possono essere semplicemente concatenati in un unico blocco: è importante che restino distinti per preservare l'ordinamento interno già calcolato da ciascun nodo. Per questo motivo, si evita l'uso di funzioni collettive, che unirebbero i dati, in favore di comunicazioni dirette punto-a-punto come **MPI_Send/MPI_Recv**. Il Master memorizza ogni array ricevuto in una struttura dedicata, un array di puntatori chiamato `partial_sa`, mantenendo così separati i contributi di ogni worker in preparazione della fase di fusione finale.

```
for (int i = 0; i < size; i++) {
    if (i == 0) {
        memcpy(partial_sa[0], local_sa, recvcnts[0] * sizeof(int));
    } else {
        MPI_Recv(partial_sa[i], recvcnts[i], MPI_INT, i, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
    }
}

iterative_merge(*global_text, partial_sa, recvcnts, size, total_n, *global_sa);

for (int i = 0; i < size; i++) {
    free(partial_sa[i]);
}
free(partial_sa);
free(recvcnts);
free(displs);
} else {
    MPI_Send(local_sa, my_size, MPI_INT, 0, 0, MPI_COMM_WORLD);
}

MPI_Barrier(MPI_COMM_WORLD);
```

Figura 12: Estratto della funzione `gather_and_merge`

3.3 Merge Iterativo: Tournament Merge

Una volta che il nodo Master ha acquisito i Suffix Array parziali mantenendoli distinti, si presenta il problema di fonderli in un'unica struttura ordinata globale. Invece di adottare un approccio ingenuo che confronti simultaneamente le teste di tutte le liste con una conseguente complessità elevata, viene adottata una strategia di **fusione gerarchica a torneo**, implementata nella funzione `iterative_merge`.

Questa funzione gestisce la logica dei round successivi, riducendo progressivamente il numero di liste attive. L'algoritmo procede accoppiando e fondendo gli array disponibili:

1. **Round 1:** si fondono le coppie adiacenti $SA_0 + SA_1 \rightarrow SA_{01}, SA_2 + SA_3 \rightarrow SA_{23}$
2. **Round successivi:** si fondono i risultati del livello precedente $SA_{01} + SA_{23} \rightarrow SA_{0123}$ fino a ottenere un unico Suffix Array globale. Nel caso in cui le liste siano dispari in un determinato round, l'ultimo array viene semplicemente promosso al livello successivo senza operazioni.

Oltre alla gestione dei round, l'implementazione di `iterative_merge` incorpora meccanismi critici per la stabilità e l'efficienza su grandi dataset:

- **Validazione Preliminare:** prima dell'avvio, viene eseguita una verifica di integrità confrontando la somma delle dimensioni dei chunk ricevuti con la dimensione totale attesa del file.
- **Gestione Dinamica dei Buffer:** per tracciare l'evoluzione del torneo, vengono utilizzati array di supporto `buffers` che mantengono i puntatori ai Suffix Array attivi, consentendo di gestire i passaggi di turno manipolando i puntatori anziché copiare dati.
- **Deallocazione Greedy:** per mitigare l'occupazione di memoria, i buffer di input di ogni coppia vengono liberati immediatamente dopo essere stati fusi nel nuovo array, garantendo che la memoria occupata sia limitata ai soli dati strettamente necessari per il livello corrente.

```

void iterative_merge(int *text, int **partial_sa, int *sizes, int num_chunks, long total_n, int *result){

    long total_check = 0;
    for (int i = 0; i < num_chunks; i++) {
        total_check += sizes[i];
    }

    if (total_check != total_n) {
        fprintf(stderr, "WARNING: Somma sizes (%ld) ≠ total_n (%ld)\n",
            total_check, total_n);
    }

    int **buffers = (int **)malloc(num_chunks * sizeof(int *));
    int *current_sizes = (int *)malloc(num_chunks * sizeof(int));

    if (!buffers || !current_sizes) {
        fprintf(stderr, "ERRORE: Impossibile allocare buffer per merge\n");
        return;
    }

    for (int i = 0; i < num_chunks; i++) {
        buffers[i] = partial_sa[i];
        current_sizes[i] = sizes[i];
    }

    int active_chunks = num_chunks;
    int round = 1;

    while (active_chunks > 1) {

        int new_active = 0;

        for (int i = 0; i < active_chunks; i += 2) {

            if (i + 1 < active_chunks) {

                int size_merged = current_sizes[i] + current_sizes[i+1];
                int *merged = (int *)malloc(size_merged * sizeof(int));

                if (!merged) {
                    fprintf(stderr, "ERRORE: Impossibile allocare memoria per merge\n");
                    fprintf(stderr, "Richiesti: %d bytes\n", size_merged * (int)sizeof(int));
                    for (int j = 0; j < new_active; j++) {
                        if (round > 1) free(buffers[j]);
                    }
                    free(buffers);
                    free(current_sizes);
                    return;
                }

                merge_two_suffix_arrays(text, buffers[i], current_sizes[i], buffers[i+1], current_sizes[i+1], merged);

                if (round > 1) {
                    free(buffers[i]);
                    free(buffers[i+1]);
                }

                buffers[new_active] = merged;
                current_sizes[new_active] = size_merged;
                new_active++;

            } else {

                buffers[new_active] = buffers[i];
                current_sizes[new_active] = current_sizes[i];
                new_active++;

            }

        }

        active_chunks = new_active;
        round++;

    }

    memcpy(result, buffers[0], total_n * sizeof(int));

    if (round > 1) {
        free(buffers[0]);
    }
    free(buffers);
    free(current_sizes);
}

```

Figura 13: Codice della funzione `iterative_merge`

L'operazione fondamentale alla base del torneo è la fusione binaria, realizzata dalla funzione **merge_two_suffix_arrays**. Questa funzione utilizza la tecnica classica dei due puntatori secondo cui vi sono due indici che percorrono le liste di input e, ad ogni passo, l'algoritmo seleziona il suffisso vincente, ovvero quello con ordine lessicografico minore, e lo inserisce nell'array di destinazione, avanzando il puntatore corrispondente. Quando una delle liste è esaurita, gli elementi rimanenti dell'altra vengono aggiunti in blocco, poiché il loro ordine interno è già corretto.

```
static void merge_two_suffix_arrays(int *text, int *sa1, int size1, int *sa2, int size2, int *result){
    int i = 0;
    int j = 0;
    int k = 0;

    while (i < size1 && j < size2) {

        int idx1 = sa1[i];
        int idx2 = sa2[j];

        int cmp = compare_suffixes(text, idx1, idx2);

        if (cmp <= 0) {
            result[k++] = sa1[i++];
        } else {
            result[k++] = sa2[j++];
        }
    }

    while (i < size1) {
        result[k++] = sa1[i++];
    }

    while (j < size2) {
        result[k++] = sa2[j++];
    }
}
```

Figura 14: Codice della funzione `merge_two_suffix_array`

La decisione su quale suffisso sia minore non avviene confrontando gli indici numerici, ma analizzando i caratteri a cui essi puntano. La funzione **compare_suffixes** esegue questo compito accedendo direttamente all'array `global_text`, centralizzato sul Rank 0 nella fase precedente. Dato che questa funzione viene invocata all'interno del ciclo più interno del merge è stata dichiarata `static inline`, così da suggerire al compilatore di espandere il codice direttamente nel punto di chiamata, eliminando l'overhead dello stack frame e massimizzando le prestazioni della CPU.

```
static inline int compare_suffixes(int *text, int idx1, int idx2) {
    int pos = 0;
    int c1, c2;

    while ((c1 = text[idx1 + pos]) == (c2 = text[idx2 + pos]) && c1 >= 0) {
        pos++;
    }

    return (c1 > c2) - (c1 < c2);
}
```

Figura 15: Codice della funzione `compare_suffixes`

3.4 Post-Processing Sequenziale

Conclusa la fase di integrazione, il nodo `Master Rank 0` dispone dell'intera struttura dati ordinata `global_sa` e del testo completo. Le fasi conclusive della pipeline, ovvero la costruzione dell'Array LCP e l'individuazione della *Longest Repeated Substring*, vengono eseguite in modalità sequenziale riutilizzando le funzioni ottimizzate `getHeight` e `findLRS` già sviluppate.

Questa scelta progettuale è giustificata da un'analisi critica della natura degli algoritmi coinvolti. Il calcolo dell'LCP mediante l'algoritmo di Kasai e la scansione lineare per l'LRS sono operazioni a complessità lineare $O(n)$. Quindi, il costo di processamento per ogni singolo elemento è ridotto a poche istruzioni di CPU per cui, parallelizzare queste fasi in un ambiente a memoria distribuita risulterebbe controproducente per due motivi principali:

- **Overhead di Comunicazione:** la decomposizione del calcolo dell'LCP richiederebbe frequenti sincronizzazioni tra i processi per gestire i suffissi che attraversano i confini dei chunk locali. Data la leggerezza delle operazioni di calcolo, la latenza di rete introdotta dallo scambio di messaggi dominerebbe il tempo di esecuzione, annullando qualsiasi beneficio derivante dal parallelismo.
- **Dipendenze dei Dati:** l'algoritmo di Kasai sfrutta una proprietà iterativa in quanto il valore corrente h dipende dal valore calcolato nell'iterazione precedente e ciò impone una forte sequenzialità logica. Spezzare questa catena di dipendenze richiederebbe algoritmi paralleli complessi e meno efficienti, il cui impiego non sarebbe giustificato poiché queste fasi incidono in modo marginale sul tempo totale rispetto all'oneroso ordinamento del Suffix Array.

3.5 Flusso di Esecuzione

Il flusso logico dell'implementazione MPI può essere schematizzato nei seguenti passaggi operativi:

- **Inizializzazione MPI:** chiamata a `MPI_Init` e determinazione di rank e dimensione del comunicatore.
- **Lettura e Sort Locale** (Parallelo): ogni processo esegue `read_and_sort_chunk()` per leggere il proprio chunk, ordinarlo localmente con `suffixSort()` e globalizzare gli indici.
- **Gather e Merge** (Centralizzato): il `Rank 0` esegue `gather_and_merge()` per ricostruire il testo globale tramite `MPI_Gatherv`, raccogliere i Suffix Array parziali tramite `MPI_Recv` e li fonde con `iterative_merge()`.
- **Post-Processing** (Sequenziale): il `Rank 0` calcola LCP con `getHeight()` e identifica LRS con `findLRS()`.
- **Finalizzazione MPI:** sincronizzazione finale con `MPI_Barrier` e chiamata a `MPI_Finalize`.

CAPITOLO 4: Implementazione Parallela con CUDA

L'implementazione CUDA rappresenta un approccio alternativo alla parallelizzazione, sfruttando l'architettura many-core delle GPU NVIDIA per accelerare operazioni massivamente parallele. A differenza dell'approccio MPI che distribuisce il carico su più nodi CPU, CUDA mira a sfruttare le migliaia di core della GPU per eseguire simultaneamente operazioni identiche su grandi quantità di dati. Tuttavia, l'algoritmo di Manber-Myers presenta caratteristiche intrinseche che ne limitano la parallelizzabilità su GPU, infatti, il confronto lessicografico dei suffissi richiede accessi irregolari alla memoria e dipendenze sequenziali difficilmente mappabili sull'architettura SIMD della GPU. Per queste ragioni, l'implementazione CUDA adotta una strategia ibrida in cui le fasi più complesse, come costruzione Suffix Array e calcolo LCP, rimangono sulla CPU, mentre la GPU viene impiegata esclusivamente per operazioni altamente parallelizzabili come la ricerca del massimo nell'LCP Array.

Il progetto si articola su due file principali:

- **cuda_main.cu**: gestisce il flusso di esecuzione ad alto livello, invocando le funzioni di lettura del file, costruzione del Suffix Array su CPU, riutilizzando `suffixSort()` della versione sequenziale, calcolo LCP su CPU con `getHeight()` e la ricerca LRS accelerata su GPU con `findLRSCUDA()`.
- **cuda_suffix_array.cu**: contiene l'implementazione del kernel GPU `reduce_max_kernel` che esegue la riduzione parallela per trovare il massimo in un array e la funzione host `findLRSCUDA()` che coordina i trasferimenti di memoria tra CPU e GPU, lancia il kernel e finalizza il risultato.

4.1 Architettura Ibrida CPU-GPU

Come già anticipato, l'implementazione CUDA non parallelizza l'intero algoritmo ma si concentra su una singola operazione altamente parallelizzabile, ovvero la ricerca del massimo nell'LCP Array per determinare la lunghezza della **Longest Repeated Substring**. Le fasi di costruzione del Suffix Array e calcolo dell'LCP Array sono eseguite sequenzialmente sulla CPU utilizzando le stesse funzioni della versione sequenziale. Questa scelta è motivata da considerazioni algoritmiche e architetturali:

- **Dipendenze forti e serialità**: l'algoritmo di Kasai per la costruzione dell'LCP è intrinsecamente sequenziale perché sfrutta il Lemma di Kasai, il quale lega il calcolo dell'elemento i -esimo al risultato dell'iterazione precedente $i-1$. Questa dipendenza dai dati impedisce l'esecuzione parallela dei thread, poiché ogni thread dovrebbe attendere il completamento del precedente, serializzando di fatto l'esecuzione sulla GPU e rendendola enormemente più lenta della CPU.
- **Sincronizzazione e divergenza**: sebbene l'ordinamento dei suffissi possa teoricamente essere parallelizzato, l'algoritmo di Manber-Myers procede per stadi iterativi. Inoltre, il confronto tra stringhe nel `qsort` richiede cicli while di lunghezza variabile per ogni coppia di suffissi. Su GPU questa variabilità causerebbe la

terminazione anticipata di alcuni thread, lasciandoli inattivi e riducendo drasticamente l'utilizzo delle unità di calcolo

- **Overhead di trasferimento:** le GPU sono collegate alla CPU tramite il bus PCIe, che ha una latenza e una banda passante limitate rispetto alla memoria interna. Per algoritmi come il sorting, che richiedono pochi calcoli per ogni byte letto, il tempo speso a trasferire i dati di input e output tra Host (CPU) e Device (GPU) rischierebbe di annullare o addirittura superare il guadagno ottenuto dal calcolo parallelo, specialmente per le dimensioni dei dataset in esame.

Di conseguenza, l'implementazione CUDA si limita ad accelerare la fase di ricerca LRS, che consiste in una semplice riduzione parallela per trovare il massimo in un array. Questa operazione è ideale per l'architettura GPU poiché non vi sono dipendenze tra elementi e gli accessi alla memoria sono regolari.

4.2 Riduzione Parallela su GPU

La funzione **findLRSCUDA()** coordina l'esecuzione della riduzione parallela sulla GPU. La procedura si articola in diverse fasi:

1. **Allocazione Memoria GPU:** vengono allocati due array sulla Device Memory, ovvero la memoria globale della GPU.
 - `d_lcp`: copia dell'LCP Array di dimensione N, allocato con **cudaMalloc**
 - `d_partial_max`: array di output che conterrà i massimi parziali di ciascun blocco, di dimensione `blocks`
2. **Trasferimento Host → Device:** l'LCP Array calcolato sulla CPU viene copiato nella memoria GPU tramite **cudaMemcpy** con direttiva `cudaMemcpyHostToDevice`. Questo trasferimento rappresenta un overhead fisso che non scala con il parallelismo.
3. **Configurazione dei Parametri di Lancio:** il kernel viene lanciato con una griglia bidimensionale definita da:
 - `threads = BLOCK_SIZE` che è il numero di thread per blocco, definito a *compile-time*, che assume come valore di default 256.
 - `blocks = (N + BLOCK_SIZE - 1) / BLOCK_SIZE` che è il numero di blocchi necessari per coprire l'intero array. Questa formula implementa una divisione intera con arrotondamento per eccesso, garantendo che vengano allocati blocchi sufficienti anche quando N non è un multiplo esatto della dimensione del blocco. Ogni blocco elabora `BLOCK_SIZE` elementi dell'array e il numero totale di thread lanciati è `blocks × threads`. Poiché questo valore può essere leggermente superiore a N, il kernel include un controllo dei limiti.

La macro `BLOCK_SIZE` può essere ottimizzata per diversi input ma i valori testati sono 32, 64, 128, 256, 512.

4. **Lancio del Kernel:** il kernel `reduce_max_kernel` viene lanciato con la sintassi:

```
reduce_max_kernel<<<blocks, threads, threads * sizeof(int)>>>(d_lcp, d_partial_max, n)
```

Il terzo parametro `threads * sizeof(int)` specifica la quantità di memoria condivisa allocata dinamicamente per ogni blocco.

5. **Trasferimento Device → Host:** dopo l'esecuzione del kernel, l'array `d_partial_max`, contenente i massimi parziali di ciascun blocco, viene copiato dalla GPU alla CPU in un array temporaneo `h_partial_max` tramite `cudaMemcpy` con direttiva `cudaMemcpyDeviceToHost`.

6. **Riduzione Finale sulla CPU:** poiché `reduce_max_kernel` produce un massimo per blocco, è necessaria una riduzione finale sui `blocks` risultati parziali. Questa viene eseguita sequenzialmente sulla CPU con un semplice ciclo che trova il massimo di `h_partial_max`.

```
int findLRSCUDA(int *lcp, int n) {
    int threads = BLOCK_SIZE;
    int blocks = (n + threads - 1) / threads;

    int *d_lcp, *d_partial_max;
    CUDA_CHECK(cudaMalloc(&d_lcp, n * sizeof(int)));
    CUDA_CHECK(cudaMalloc(&d_partial_max, blocks * sizeof(int)));

    CUDA_CHECK(cudaMemcpy(d_lcp, lcp, n * sizeof(int), cudaMemcpyHostToDevice));

    reduce_max_kernel<<<blocks, threads, threads * sizeof(int)>>>(d_lcp, d_partial_max, n);

    int *h_partial_max = (int*)malloc(blocks * sizeof(int));
    CUDA_CHECK(cudaMemcpy(h_partial_max, d_partial_max, blocks * sizeof(int), cudaMemcpyDeviceToHost));

    int max_lcp = 0;
    for(int i = 0; i < blocks; i++) {
        if(h_partial_max[i] > max_lcp) max_lcp = h_partial_max[i];
    }

    free(h_partial_max);
    CUDA_CHECK(cudaFree(d_lcp));
    CUDA_CHECK(cudaFree(d_partial_max));
    return max_lcp;
}
```

Figura 16: Codice della funzione `findLRSCUDA`

4.3 Implementazione del Kernel

Il cuore dell'accelerazione su GPU risiede nel kernel `reduce_max_kernel`, il quale implementa una strategia di riduzione binaria parallela. Dichiarato con il qualificatore `__global__`, questo kernel funge da punto di ingresso per l'esecuzione sul device, venendo invocato dalla CPU (Host) ma eseguito in parallelo da migliaia di thread sulla GPU.

La logica di esecuzione si articola in due fasi distinte: il caricamento cooperativo in memoria condivisa e la riduzione algoritmica vera e propria.

1. Caricamento cooperativo e memoria condivisa

La prima operazione critica è il trasferimento dei dati dalla Global Memory alla Shared Memory. Ogni thread del blocco partecipa a questo processo caricando un singolo elemento:

- Viene calcolato l'**indice globale** `i` (`blockIdx.x * blockDim.x + threadIdx.x`), che identifica la posizione univoca dell'elemento nell'array di input completo.
- Viene calcolato l'**indice locale** `tid` (`threadIdx.x`), che rappresenta la posizione del thread all'interno del blocco corrente (da 0 a `BLOCK_SIZE-1`).

Per gestire correttamente array di dimensioni arbitrarie, il caricamento include un controllo dei limiti: se l'indice globale `i` rientra nei limiti dell'array, quindi `i < n`, il thread copia il valore corrispondente in `sdata[tid]`. In caso contrario, assegna all'elemento condiviso il valore neutro 0. Al termine del caricamento, viene invocata la primitiva di sincronizzazione `__syncthreads()`. Questa barriera garantisce che nessun thread inizi la fase di calcolo prima che tutti gli altri thread del blocco abbiano completato la scrittura in `sdata`. Senza questa precauzione, si rischierebbe che un thread tenti di leggere un valore non ancora caricato dal suo vicino.

2. Algoritmo di Riduzione Binaria

Una volta che i dati sono stabili in Shared Memory, inizia la fase di riduzione. L'algoritmo non utilizza un approccio lineare, ma procede per iterazioni logaritmiche dimezzando progressivamente il numero di thread attivi.

Il ciclo `for` è governato da uno stride `s`, inizializzato a metà della dimensione del blocco (`blockDim.x / 2`) e dimezzato ad ogni iterazione tramite operatore di bit-shift `s >>= 1`.

In ogni passo del ciclo:

- Sono attivi solo i thread con indice locale `tid < s`.
- Ciascun thread attivo confronta il proprio elemento `sdata[tid]` con l'elemento distante `s` posizioni, ovvero `sdata[tid + s]`.
- Se l'elemento distante è maggiore, esso sovrascrive quello locale.

- Al termine di ogni operazione di confronto e scambio, una nuova barriera `__syncthreads()` assicura che i risultati parziali siano visibili a tutti i thread prima di procedere al livello successivo.

```
__global__ void reduce_max_kernel(int *d_input, int *d_output, int n) {
    extern __shared__ int sdata[];
    unsigned int tid = threadIdx.x;
    unsigned int i = blockIdx.x * blockDim.x + threadIdx.x;

    sdata[tid] = (i < n) ? d_input[i] : 0;
    __syncthreads();

    for (unsigned int s = blockDim.x / 2; s > 0; s >>= 1) {
        if (tid < s) {
            if (sdata[tid + s] > sdata[tid]) {
                sdata[tid] = sdata[tid + s];
            }
        }
        __syncthreads();
    }

    if (tid == 0) d_output[blockIdx.x] = sdata[0];
}
```

Figura 17: Codice del kernel `reduce_max_kernel`

4.4 Flusso di Esecuzione

Il flusso logico dell'implementazione CUDA può essere schematizzato nei seguenti passaggi operativi:

- **Lettura File** (CPU): caricamento del dataset binario tramite `readBinaryFile()` e conversione in array di interi text.
- **Costruzione Suffix Array** (CPU): ordinamento dei suffissi tramite `suffixSort()`, algoritmo di Manber-Myers sequenziale.
- **Calcolo LCP Array** (CPU): costruzione dell'array `lcp` utilizzando `getHeight()`, algoritmo di Kasai).
- **Ricerca LRS** (GPU): invocazione di `findLRSCUDA()` che alloca memoria GPU, trasferisce l'LCP Array, lancia il kernel di riduzione parallela, trasferisce i risultati parziali e finalizza il massimo su CPU.

CAPITOLO 5: Test e Analisi

Il presente capitolo è dedicato alla validazione sperimentale delle soluzioni proposte, con l'obiettivo di quantificare le prestazioni delle implementazioni parallele MPI e CUDA rispetto alla baseline sequenziale. L'analisi si articola in tre fasi distinte: la definizione di una metodologia per la generazione di dataset riproducibili, la descrizione dell'ambiente hardware e software di test e infine la discussione critica dei risultati ottenuti.

5.1 Metodologia Di Generazione Del Dataset Di Test

Per valutare le prestazioni e la correttezza delle implementazioni sequenziali e parallele dell'algoritmo di Manber-Myers, è stato necessario generare un dataset di stringhe binarie pseudo-casuali di dimensioni crescenti 1 MB, 50 MB, 100 MB, 200 MB, 500 MB. L'implementazione di tale procedura è incapsulata nel modulo `string_generator.c`, il quale provvede alla creazione e al salvataggio dei file binari risultanti all'interno della directory di progetto `Data/`.

La scelta del generatore di numeri pseudo-casuali si è rivelata un aspetto critico. È stata preferita la generazione pseudo-casuale rispetto a quella realmente casuale principalmente per garantire la riproducibilità degli esperimenti. Infatti, il generatore è stato implementato in C utilizzando un **seed fisso** (12345678901234567) in modo da produrre esattamente gli stessi file binari e consentendo così un confronto equo tra le diverse versioni dell'algoritmo.

Tuttavia, generatori inadeguati possono introdurre pattern ripetitivi che falsano l'analisi della Longest Repeated Substring. Inizialmente, per la generazione dei dati è stata utilizzata la funzione `rand()` della libreria standard C. Questa funzione implementa un Generatore Congruenziale Lineare (LCG), definito dalla ricorrenza: $X_{n+1} = (a \cdot X_n + c) \pmod{m}$

Nell'implementazione specifica dei sistemi Windows, vengono restituiti solo i 15 bit più significativi con una conseguente limitazione del periodo effettivo della sequenza di byte a circa 2^{24} valori, ovvero 16.777.216 byte (~16 MB). Questa limitazione ha causato un errore significativo nei risultati, infatti, generando file superiori a 16 MB, la sequenza di byte iniziava a ripetersi ciclicamente. Durante i test sul file da 500 MB, l'algoritmo ha rilevato una LRS di oltre 8 milioni di caratteri, un valore statisticamente impossibile per una stringa casuale, che ha motivato l'abbandono di `rand()` in favore dell'algoritmo **xorshift64***.

5.1.1 xorshift64*

Per superare i limiti dell'LCG, è stato implementato l'algoritmo **xorshift64***, una variante dei generatori *xorshift* introdotti da **George Marsaglia** nel 2003 e successivamente perfezionati da **Sebastiano Vigna** nel 2016.

I generatori xorshift basano il loro funzionamento su operazioni lineari quali lo **shift** e lo **XOR**. La variante **xorshift64*** introduce uno step di permutazione non lineare, infatti, l'algoritmo esegue le trasformazioni lineari dello stato e successivamente moltiplica il risultato per una costante invertibile M_{32} prima di restituirlo.

La struttura dell'algoritmo è la seguente:

- **Aggiornamento dello stato:** inizialmente si applica un'operazione di XOR tra lo stato corrente e la sua versione traslata a destra di 12 posizioni. Il risultato viene successivamente combinato con sé stesso traslato a sinistra di 25 posizioni, concludendo con un ulteriore shift a destra di 27. Questa specifica terna di parametri è stata determinata per massimizzare il mescolamento dei bit e garantire il periodo completo del generatore.

$$x \leftarrow x \oplus (x \gg 12)$$

$$x \leftarrow x \oplus (x \ll 25)$$

$$x \leftarrow x \oplus (x \gg 27)$$

- **Output permutazione:** lo stato aggiornato viene moltiplicato per la costante M_{32} , un moltiplicatore dispari che introduce una componente di non-linearità, necessaria per rompere le correlazioni lineari introdotte dagli shift, rendendo la sequenza meno prevedibile. Questa operazione genera un *avalanche effect* (effetto valanga) che diffonde l'entropia uniformemente su tutti i bit dell'output, correggendo la debolezza statistica dei singoli bit. Infine, poiché la costante è dispari e quindi invertibile nell'aritmetica del calcolatore, l'operazione agisce come una permutazione che rimescola i valori senza mai sovrapporli, garantendo che il periodo del generatore resti intatto ed eliminando al contempo pattern prevedibili che causerebbero il fallimento dei test.

$$\text{return } x * 0x2545F4914F6CDD1D$$

L'adozione di xorshift64* ha garantito due proprietà fondamentali per il testing del Suffix Array:

- **Periodo Esteso:** il generatore ha un periodo di $2^{64} - 1$, garantendo l'assenza di ripetizioni cicliche anche generando file di grandi dimensioni, risolvendo il problema riscontrato con rand().
- **Efficienza:** utilizzando solo operazioni bitwise e una singola moltiplicazione, il generatore è estremamente veloce e non introduce overhead significativi nella fase di preprocessing dei dati.

La validità del dataset generato con xorshift64* è stata confermata analizzando i risultati della Longest Repeated Substring (LRS). Per una stringa casuale di lunghezza n con alfabeto di dimensione $\sigma = 256$, la lunghezza attesa della LRS segue la legge logaritmica:

$$LRS \approx 2 \cdot \log_{\sigma}(n)$$

La tabella di seguito mostra i risultati ottenuti. Essi risultano perfettamente coerenti con le previsioni teoriche, confermando l'assenza di pattern artificiali e l'assenza di bias statistici nel generatore.

Dimensione File	Lunghezza (n)	LRS Atteso Teorico	LRS Misurato
1 MB	49.932 byte	3-4	4
50 MB	2.496.609 byte	4-5	5
100 MB	4.993.219 byte	5-6	5
200 MB	9.986.438 byte	5-6	5
500 MB	24.966.095 byte	6-7	6

Tabella 1: Validazione Statistica del Dataset Generato con xorshift64*

La selezione dell'algoritmo **xorshift64*** è stata guidata dalla necessità di garantire una robustezza statistica, privilegiando la qualità della distribuzione dei singoli bit rispetto all'estensione del periodo. Sebbene esistano generatori che offrono periodi di dimensione maggiore, la priorità del progetto è stata quella di evitare l'introduzione di qualsiasi artefatto o pattern strutturale che potesse inficiare la generazione dei dati di test. In quest'ottica, **xorshift64*** si distingue per l'impiego di una moltiplicazione non lineare per la costante M_{32} , che mitiga le debolezze tipiche dei generatori puramente lineari, particolarmente evidenti nei bit meno significativi, assicurando un'elevata entropia su tutto lo spettro dell'output. Infine, l'analisi dimensionale conferma che il periodo del generatore, pur non essendo il massimo teorico esistente, risulta superiore di diversi ordini di grandezza rispetto alla massima sequenza richiesta dal progetto, rendendo il rischio di collisioni del tutto trascurabile.

5.1.2 Analisi e Gestione dell'Overhead

L'implementazione del modulo **string_generator.c** ha richiesto una gestione attenta delle risorse per garantire il rispetto dei vincoli di memoria previsti e l'efficienza della generazione. Sono state affrontate due tipologie critiche di overhead:

1. **Overhead strutturale:** il requisito progettuale definisce le dimensioni target 1 MB, 50 MB, 100MB, 200 MB e 500 MB in termini di occupazione complessiva in RAM durante l'esecuzione dell'algoritmo e non come dimensione del file su disco.

L'algoritmo di Manber-Myers, integrato con il calcolo dell'LCP, richiede l'allocazione di molteplici strutture ausiliarie oltre alla stringa stessa. Per ogni carattere della stringa di input, il sistema deve allocare spazio per:

1. **Testo convertito** (`int *text`): 4 byte per carattere
2. **Suffix Array** (`int *sa`): 4 byte per elemento.
3. **LCP Array** (`int *height`): 4 byte per elemento.
4. **Rank Array** (`int *rank`): 4 byte per elemento.
5. **Struttura Suffix** (durante il sort): 12 byte per elemento.
6. **Array ausiliari vari** (`cnt`, `next_arr`, `bh`, `b2h`): circa 4-5 byte.

Sommando questi contributi, l'algoritmo consuma circa **26 byte** di memoria dinamica per ogni singolo byte di carattere presente nel file di input. Considerando l'ulteriore overhead introdotto

dal sistema operativo per la gestione dell'allocazione dinamica e l'allineamento della memoria, è stato calcolato un **fattore di overhead** conservativo pari a **21**. Di conseguenza, la dimensione fisica del file binario generato su disco D_{file} viene calcolata inversamente rispetto al target di memoria M_{target} secondo la formula:

$$D_{file} = \frac{M_{target} \times 1024 \times 1024}{Overhead}$$

Per questo motivo, il test con occupazione in memoria di 500 MB corrisponde alla generazione fisica di un file di soli ~23.81 MB (24.966.095 byte).

2. **Overhead di Scrittura:** la scrittura sequenziale di milioni di byte comporta un significativo overhead di Input/Output dovuto alle frequenti system call al kernel. Per mitigare questo costo, è stata implementata una strategia di scrittura bufferizzata. Il generatore accumula i byte prodotti da xorshift64* in un buffer temporaneo in RAM da 1 MB e la funzione di scrittura su disco fwrite() viene invocata unicamente al riempimento del buffer. Questa tecnica riduce il numero di chiamate al sistema, rendendo il tempo di I/O trascurabile rispetto al tempo di generazione.

5.2 Experimental Setup

Hardware Specifications (Local Machine)

Gli esperimenti sono stati condotti su una macchina locale con le seguenti caratteristiche. Si noti che, sebbene la macchina disponga di 8 GB di RAM fisica, il sottosistema WSL2 utilizzato per i test MPI era limitato a circa 4 GB.

CPU	
Model	Intel(R) Core(TM) i7-8565U CPU @ 1.80GHz
Architecture	x86_64 (AMD64)
Cores	4 Physical Cores, 8 Logical Processors
Base Frequency	1.80 GHz

RAM	
Total Physical Memory	8.0 GB (7.89 GiB available to Windows)
Type/Speed	DDR4 @ 2400 MHz
WSL Allocation	~3.77 GiB (Allocated to Linux Subsystem)

GPU (CUDA Device)	
Model	NVIDIA GeForce MX130
VRAM	2048 MiB (2 GB)
Compute Capability	5.0
Driver Version	577.03

Software Environment

Il progetto utilizza un ambiente di sviluppo ibrido che prevede l'uso di Windows per l'esecuzione delle versioni CUDA e sequenziali e di WSL2 (Windows Subsystem for Linux) per l'esecuzione MPI.

Host Environment (Windows - Sequential & CUDA)	
OS	Windows 10/11 (Build 10.0.26100)
Compiler	GCC 15.2.0 (via MSYS2)
CUDA Toolkit	Version 11.8 (Build cuda_11.8.r11.8/compiler.31833905_0)
Python	3.10.11 (Scripting & Automation)

Execution Environment (WSL2 - MPI)	
OS	Ubuntu 24.04 LTS (Kernel 6.6.87.2-microsoft-standard-WSL2)
Compiler	gcc (Ubuntu 13.3.0-6ubuntu2~24.04) 13.3.0
MPI Implementation	MPICH / OpenMPI (Standard Ubuntu Repository)
Python	3.12.3

5.3 Analisi Comparativa delle Prestazioni

Questa sezione presenta l'analisi dettagliata dei risultati sperimentali ottenuti dalle tre implementazioni dell'algoritmo di Manber-Myers: sequenziale, MPI e CUDA. I test sono stati condotti su dataset di dimensioni crescenti (1 MB, 50 MB, 100 MB, 200 MB, 500 MB) utilizzando tutte le combinazioni di ottimizzazioni del compilatore (-O0, -O1, -O2, -O3), configurazioni di processi MPI (2 e 4) e dimensioni di blocco CUDA (32, 64, 128, 256, 512 thread). Per garantire l'affidabilità statistica, ogni configurazione è stata eseguita tre volte, calcolando la media aritmetica dei tempi di esecuzione. Tutti gli esperimenti hanno prodotto risultati identici in termini di correttezza, confermando la validità delle tre implementazioni attraverso il confronto del valore LRS finale.

5.3.1 Impatto delle Ottimizzazioni del Compilatore

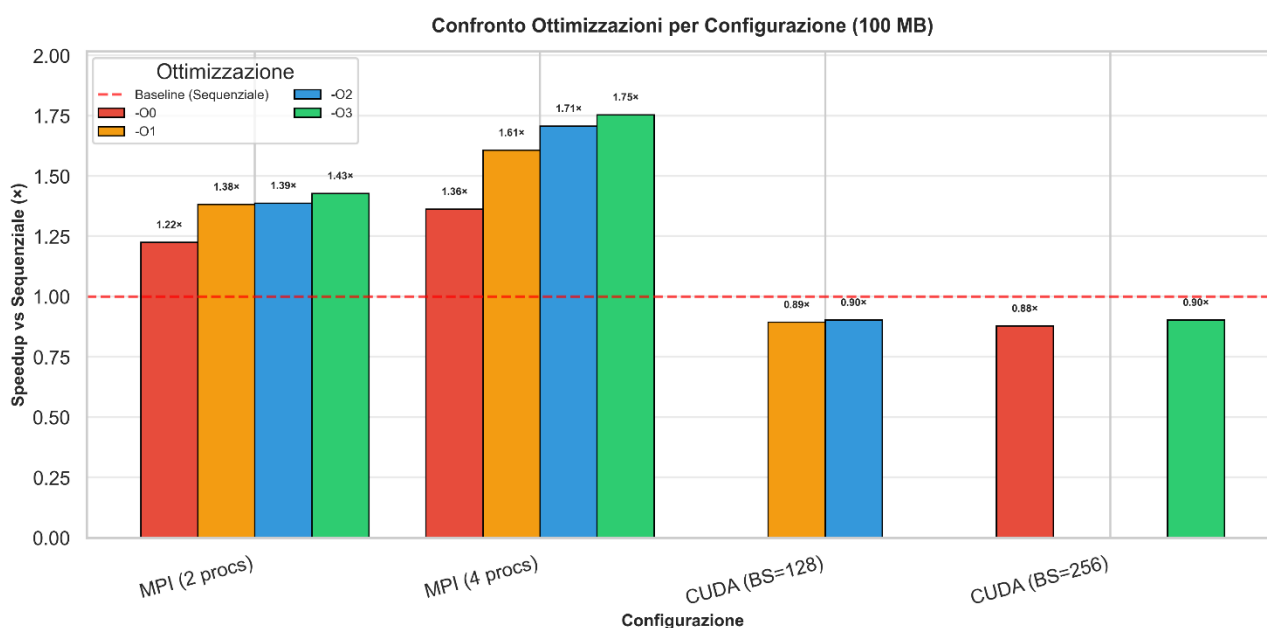


Figura 18: Confronto tra i quattro livelli di ottimizzazione del compilatore GCC sull'implementazione sequenziale con input di 100 MB. Il livello -O0 serve da baseline non ottimizzata (3.34s), mentre -O1/-O2/-O3 mostrano miglioramenti del 14-19%.

L'ottimizzazione del compilatore rappresenta un fattore determinante nelle prestazioni dell'implementazione sequenziale. La Figura 18 mostra l'impatto dei quattro livelli di ottimizzazione su un input di 100 MB. Il livello -O0 (nessuna ottimizzazione) richiede **3.34 secondi**, costituendo la baseline più lenta. L'attivazione di -O1 riduce il tempo a **2.71 secondi**, corrispondente a un miglioramento del 18.8% rispetto a -O0. Questa accelerazione deriva principalmente dall'eliminazione del codice morto, dall'ottimizzazione dei loop semplici e dalla propagazione delle costanti. I livelli -O2 e -O3 producono prestazioni pressoché identiche, rispettivamente **2.72 secondi** (18.6% di miglioramento) e **2.86 secondi** (14.5% di miglioramento). Tutte le analisi successive utilizzano -O3 come configurazione di riferimento, garantendo la massima performance del compilatore senza penalizzazioni evidenti.

5.3.2 Breakdown Temporale per Fase

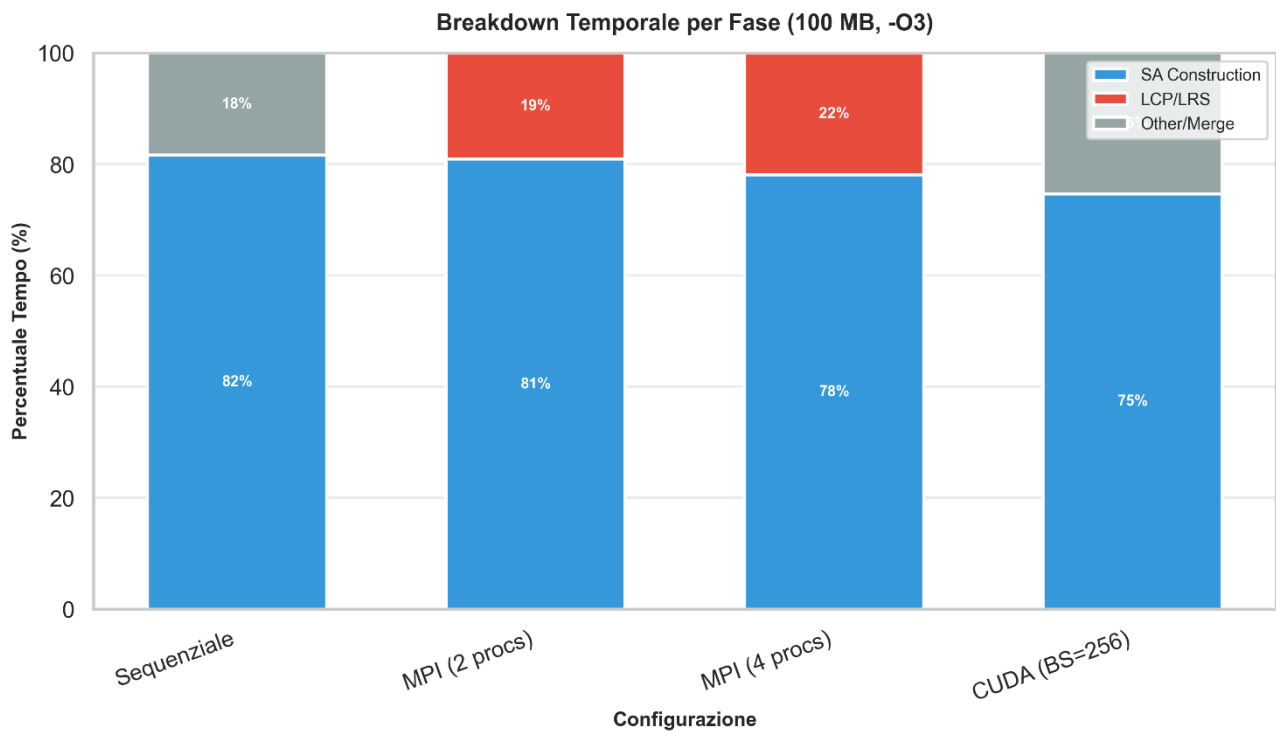


Figura 19: Distribuzione percentuale del tempo di esecuzione nelle tre fasi principali (SA Construction, LCP/LRS, Other/Merge) per le quattro configurazioni testate. La costruzione del Suffix Array domina con il 75-82% del tempo totale, confermando che rappresenta il collo di bottiglia algoritmico principale. MPI mostra una frazione crescente di tempo dedicato al merge all'aumentare del numero di processi (18% con $P=2$, 22% con $P=4$).

La Figura 19 presenta la distribuzione percentuale del tempo di esecuzione nelle tre fasi principali dell'algoritmo per input di 100 MB con ottimizzazione `-O3`. Nell'implementazione sequenziale, la costruzione del Suffix Array domina con l'82% del tempo totale (**2.33 secondi** su **2.86 secondi**), confermando che l'ordinamento dei suffissi rappresenta la fase computazionalmente più onerosa. Il calcolo dell'LCP Array richiede il 18% del tempo (**0.52 secondi**), mentre la ricerca LRS è trascurabile (**0.006 secondi**, $<0.2\%$). Questa distribuzione si mantiene stabile al variare della dimensione dell'input.

Nell'implementazione MPI con 2 processi, la distribuzione cambia leggermente: la fase di ordinamento locale rappresenta l'81% del tempo (**1.62 secondi**), mentre il 19% (**0.38 secondi**) è dedicato alle operazioni di merge e al post-processing combinato (LCP + LRS). Con 4 processi, la frazione di merge aumenta al 22% (**0.36 secondi** su **1.63 secondi** totali) nonostante il tempo assoluto di merge rimanga sostanzialmente costante. Questo fenomeno è attribuibile alla natura logaritmica del merge iterativo: il costo computazionale del merge è $O(N \log P)$, quindi aumenta moderatamente con il numero di processi, mentre il tempo di ordinamento locale $O(N/P \log N/P)$ diminuisce linearmente. L'overhead di comunicazione per il gather del testo e la raccolta dei Suffix Array parziali contribuisce a questo incremento percentuale, evidenziando i limiti della scalabilità oltre $P=4$ processi.

L'implementazione CUDA, con block size 256, mostra una distribuzione sorprendentemente simile alla versione sequenziale: il 75% del tempo (**2.36 secondi**) è

impiegato nella costruzione del Suffix Array, il 16% (**0.51 secondi**) nel calcolo dell'LCP e il 9% (**0.29 secondi**) nella ricerca LRS accelerata su GPU. Sebbene la riduzione parallela per trovare il massimo nell'LCP Array sia eseguita sulla GPU, il tempo totale di questa fase include l'overhead di trasferimento dei dati tra CPU e GPU, che costituisce circa il 70% del tempo della fase LRS. Questo overhead fisso, stimato in circa **0.20 secondi** indipendentemente dalla dimensione dell'input, neutralizza i benefici della parallelizzazione massiva offerta dalla GPU per la riduzione, spiegando perché l'implementazione CUDA non ottiene accelerazioni significative.

5.3.3 Confronto Diretto: MPI vs CUDA

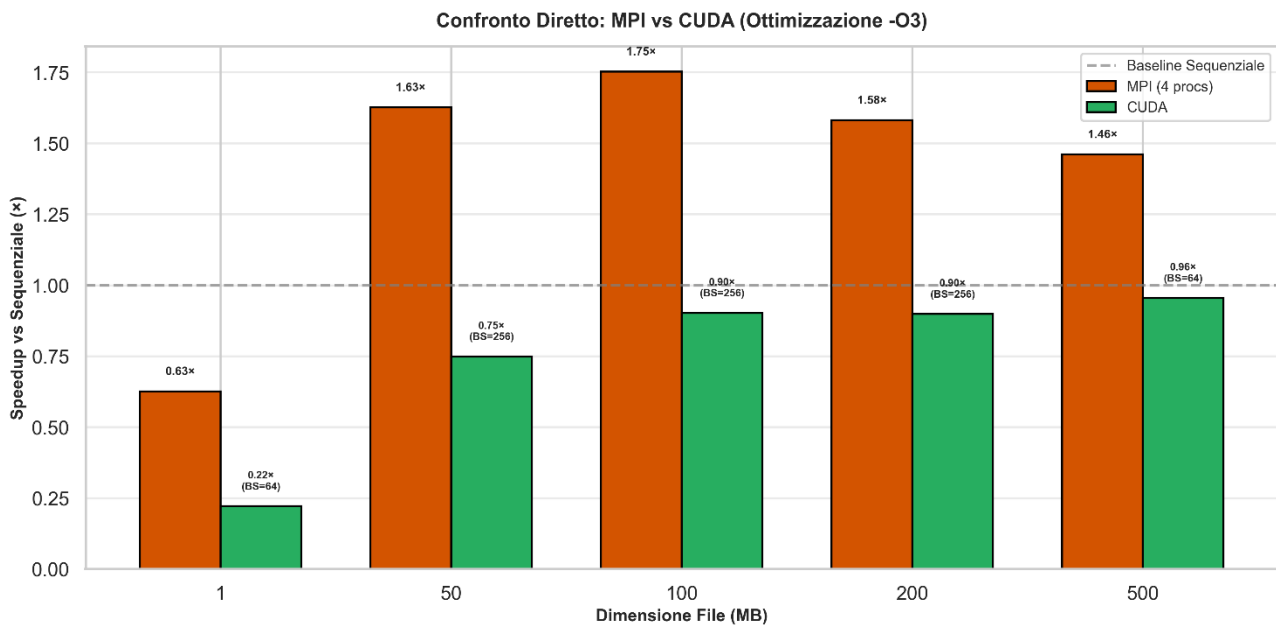


Figura 20: Speedup relativo alla baseline sequenziale per MPI (4 processi) e CUDA (miglior block size per dimensione) su tutti gli input testati. MPI raggiunge speedup significativi (1.46-1.75x) mentre CUDA rimane costantemente sotto 1.0x (0.75-0.98x), evidenziando la superiorità dell'approccio a memoria distribuita per questo problema specifico.

La Figura 20 confronta direttamente le prestazioni di MPI (4 processi) e CUDA (block size ottimale per dimensione) rispetto alla baseline sequenziale su tutti gli input testati. Su file di 50 MB, MPI raggiunge uno speedup di **1.63x**, riducendo il tempo da **1.06 secondi** a **0.65 secondi**, mentre CUDA ottiene solo **0.75x** (**1.41 secondi**), risultando addirittura più lento della versione sequenziale. Questo comportamento anomalo è dovuto all'overhead fisso di inizializzazione della GPU e ai trasferimenti di memoria che per input piccoli superano il guadagno computazionale. Su 100 MB, MPI mantiene uno speedup di **1.75x** (**1.63 secondi** contro **2.86 sequenziali**) mentre CUDA raggiunge appena **0.90x** (**3.16 secondi**). Per input di 200 MB, MPI ottiene **1.58x** (**4.03 secondi**) e CUDA **0.90x** (**7.08 secondi**). Infine, su 500 MB, MPI scende a **1.46x** (**12.22 secondi** contro **17.85 sequenziali**) mentre CUDA rimane stabile a **0.94x** (**19.06 secondi**).

Questo confronto rivela una netta superiorità dell'approccio MPI rispetto a CUDA per il problema della costruzione del Suffix Array. Le ragioni sono molteplici. In primo luogo, l'implementazione CUDA adottata accelera solo la fase di ricerca LRS, che rappresenta meno

del 2% del tempo totale, lasciando invariate le fasi dominanti (SA e LCP) che restano sulla CPU. In secondo luogo, l'overhead di trasferimento host-device introduce una penalità fissa che non scala con la dimensione dell'input, particolarmente penalizzante per file < 100 MB. Al contrario, MPI distribuisce il carico computazionale della fase più onerosa su più core CPU, ottenendo accelerazioni proporzionali al numero di processi, limitata solo dal merge finale. In terzo luogo, le performance CUDA sono ulteriormente penalizzate dall'hardware utilizzato (GeForce MX130), una GPU mobile con larghezza di banda limitata verso la memoria host, che accentua il peso relativo dei trasferimenti rispetto al calcolo.

Implementazione	Configurazione	Tempo Totale (s)	Speedup vs Seq	% Tempo SA	% Tempo Altro
Sequenziale	Baseline	2.86 s	1.00×	82%	18%
MPI	2 Processi	2.00 s	1.43×	81%	19%
MPI	4 Processi	1.63 s	1.75×	78%	22%
CUDA	Block Size 256	3.16 s	0.90×	75%	25%

Tabella 2: Confronto diretto tra le implementazioni Sequenziale, MPI e CUDA. La tabella riporta i tempi totali assoluti, lo speedup relativo rispetto alla baseline sequenziale e la ripartizione percentuale del carico di lavoro. Si evidenzia come la costruzione del Suffix Array rappresenti la fase dominante (75-82% del tempo) in tutte le configurazioni, mentre la colonna "Altro" mostra l'impatto crescente dell'overhead di gestione (Merge per MPI, trasferimenti di memoria per CUDA).

5.3.4 Effetto del Block Size su CUDA

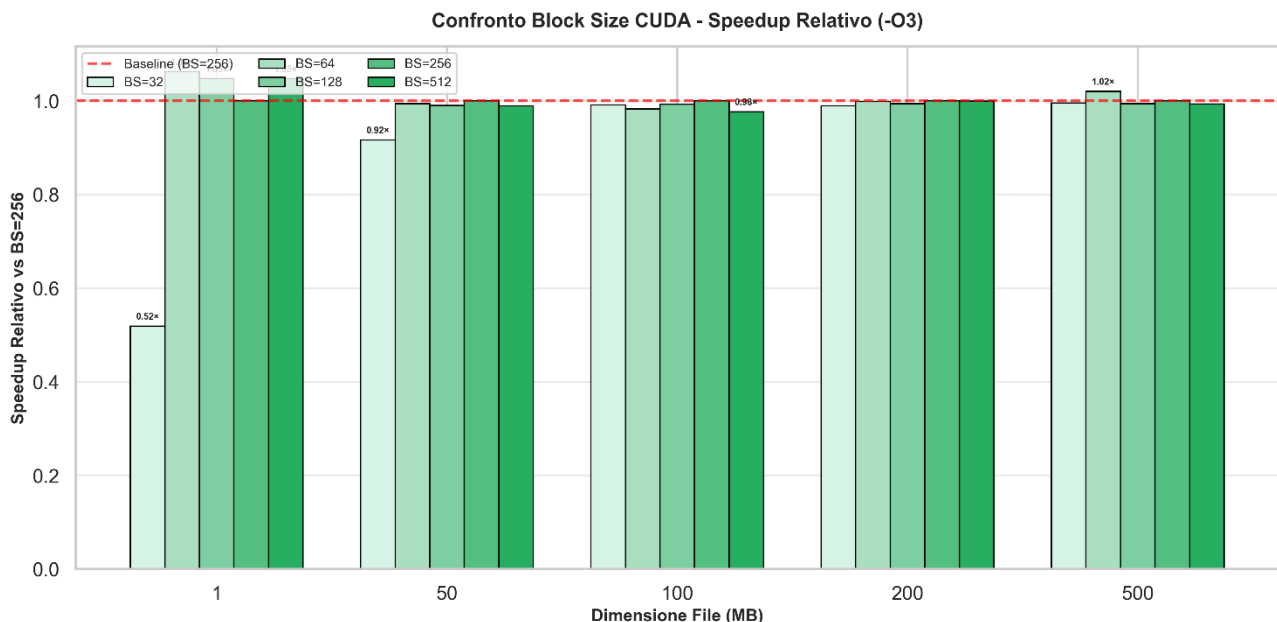


Figura 21: Speedup relativo di ciascun block size CUDA rispetto alla configurazione baseline (BS=256). Su input di 1 MB, BS=32 mostra prestazioni drasticamente inferiori (0.52×) a causa della bassa occupancy dei multiprocessori. Per input ≥ 50 MB, tutte le configurazioni convergono a prestazioni simili (±2%), indicando che il block size non è il fattore limitante.

La Figura 21 analizza l'impatto della dimensione del blocco CUDA sullo speedup relativo rispetto alla configurazione baseline (BS=256). Su input di 1 MB, le variazioni sono drastiche: BS=32 ottiene solo **0.52×** (**141 ms** contro **73 ms** del baseline), mentre BS=64, 128, 256

e 512 convergono tutti a prestazioni simili ($\sim 1.00\times$). Tale inefficienza è dovuta a una sottoutilizzazione delle risorse hardware: configurare blocchi da soli 32 thread non è sufficiente a saturare la capacità di calcolo dei multiprocessori, lasciando di conseguenza inattivi gran parte dei core della GPU. Per input più grandi (50–500 MB), le differenze si riducono drasticamente: tutti i block size producono tempi entro il 2-3% dal baseline. La Figura 22 conferma questa tendenza, mostrando che per input ≥ 100 MB il tempo di esecuzione varia tra **3.16 e 3.24 secondi** indipendentemente da BS, indicando che il kernel di riduzione non è il fattore limitante. Il collo di bottiglia risiede nelle fasi SA e LCP eseguite sequenzialmente sulla CPU, rendendo irrilevante l'ottimizzazione del block size per la fase LRS. Questa analisi suggerisce che un'implementazione CUDA realmente efficace dovrebbe parallelizzare anche le fasi di ordinamento e calcolo LCP, non solo la ricerca del massimo.

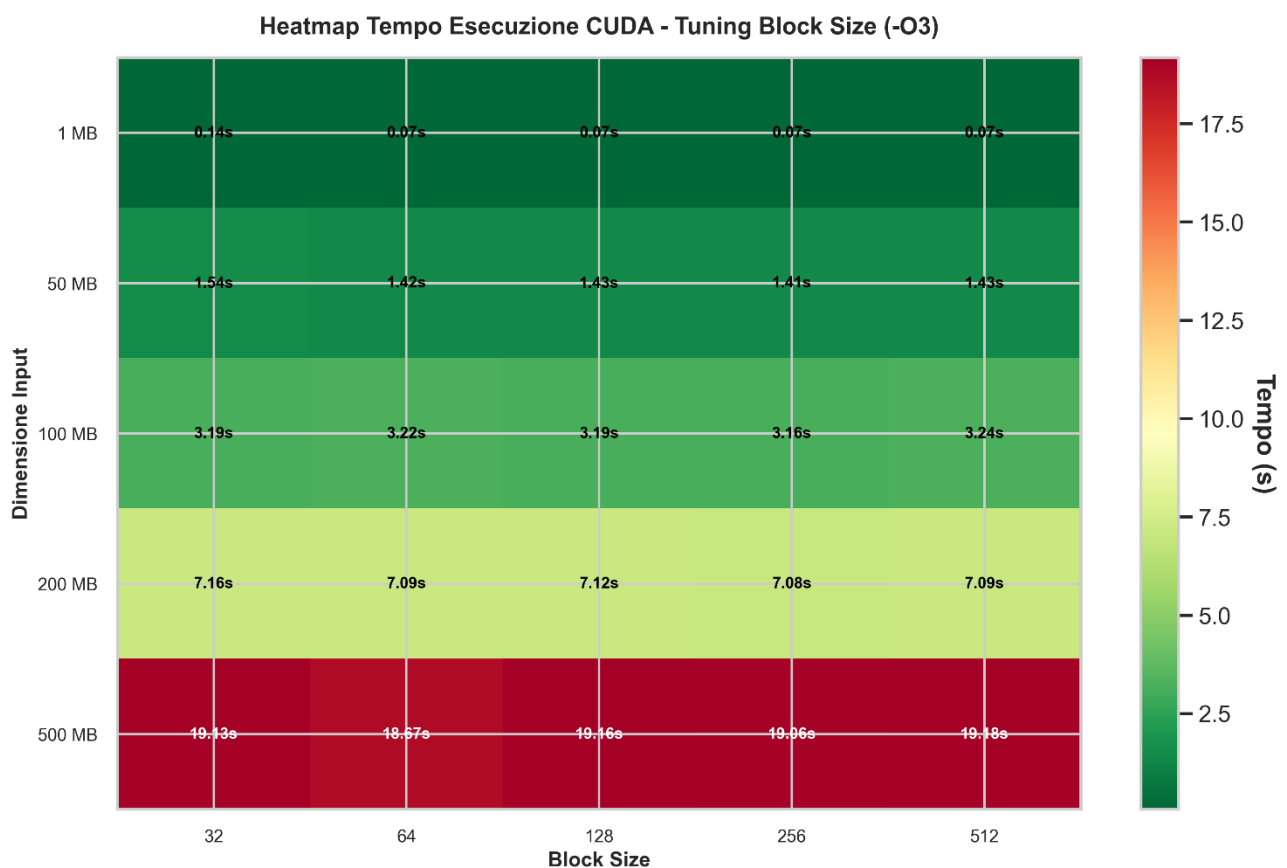


Figura 22: Heatmap dei tempi assoluti di esecuzione CUDA al variare di block size (32-512) e dimensione input (1-500 MB). Il gradiente di colore evidenzia che per input ≥ 100 MB il tempo è sostanzialmente indipendente dal block size (verde uniforme), confermando che il collo di bottiglia risiede nelle fasi SA e LCP eseguite su CPU.

5.3.5 Scalabilità MPI: Efficienza Parallela

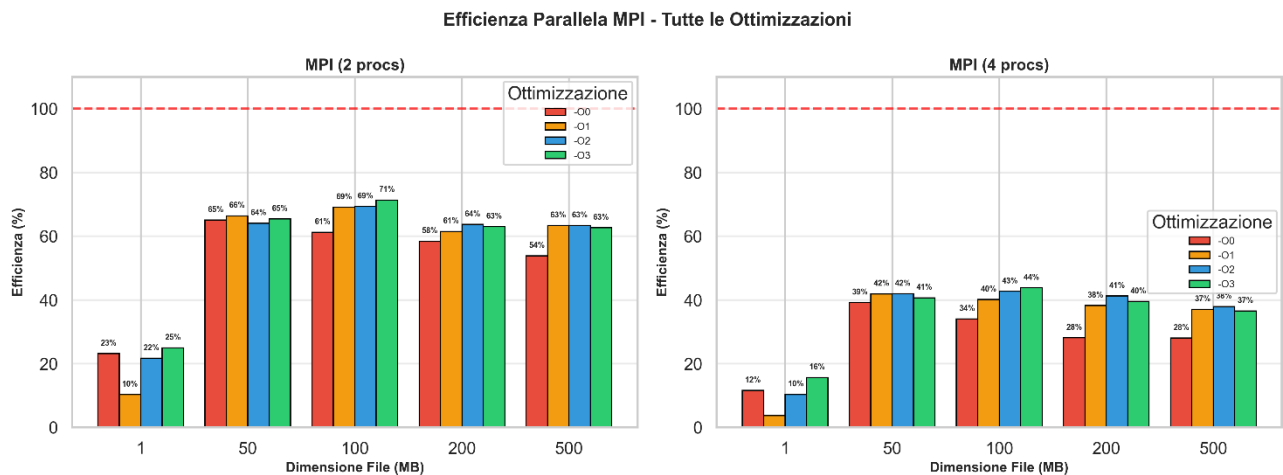


Figura 23: Efficienza parallela per MPI con 2 e 4 processi su tutti gli input e ottimizzazioni. Su input piccoli (1 MB), l'efficienza crolla a 10-25% a causa dell'overhead di comunicazione fisso. Su input grandi (≥ 100 MB), E(2) stabilizza a 63-71% ed E(4) a 40-44%, indicando che il tempo di calcolo compensa l'overhead di setup MPI.

La Figura 23 mostra l'efficienza parallela di MPI al variare del numero di processi e della dimensione dell'input. L'efficienza è definita come $E(P) = \text{Speedup}(P) / P$, dove $\text{Speedup}(P) = T_{\text{seq}} / T_{\text{par}}(P)$. Idealmente, $E(P) = 100\%$ indica scalabilità perfetta. Su input di 1 MB, l'efficienza crolla drasticamente: con 2 processi si ottiene $E(2) = 23\%$ (-O0) fino al 25% (-O3), mentre con 4 processi $E(4)$ scende a $12\text{-}16\%$. Questo fenomeno è attribuibile al fatto che l'overhead di comunicazione (setup MPI, gather, send/recv) è fisso e indipendente dalla dimensione, dominando completamente il tempo di calcolo per input piccoli. Su 50 MB, l'efficienza migliora significativamente: $E(2) = 61\text{-}71\%$ ed $E(4) = 34\text{-}44\%$, indicando che il tempo di calcolo inizia a compensare l'overhead di comunicazione. Su 100 MB, $E(2)$ stabilizza a $63\text{-}71\%$ ed $E(4)$ a $40\text{-}44\%$. Per 200 MB e 500 MB, l'efficienza si mantiene su livelli simili, confermando che MPI raggiunge un plateau di scalabilità oltre i 100 MB di input. L'efficienza relativamente bassa con 4 processi è dovuta al fatto che la frazione seriale rappresentata dal merge centralizzato sul Rank 0 (circa 20-22% del tempo totale) limita lo speedup massimo, rendendo impossibile raggiungere efficienze superiori al 50% con $P=4$. Il grafico conferma inoltre che livelli di ottimizzazione più aggressivi (-O2, -O3) non migliorano sensibilmente l'efficienza parallela, poiché questa è dominata dalla struttura algoritmica e non dal codice generato dal compilatore.

Dimensione Input	Speedup (P=2)	Efficienza (P=2)	Speedup (P=4)	Efficienza (P=4)
1 MB	0.50×	25%	0.63×	16%
50 MB	1.30×	65%	1.62×	41%
100 MB	1.43×	71%	1.75×	44%
200 MB	1.26×	63%	1.58×	40%
500 MB	1.25×	63%	1.46×	37%

Tabella 3: Dettaglio dello speedup e dell'efficienza parallela per 2 e 4 processi al variare della dimensione del dataset. I dati mostrano chiaramente la correlazione tra dimensione dell'input e rendimento.

5.3.6 Speedup Aggregato per Tutte le Ottimizzazioni

Speedup vs Sequenziale - Tutte le Ottimizzazioni

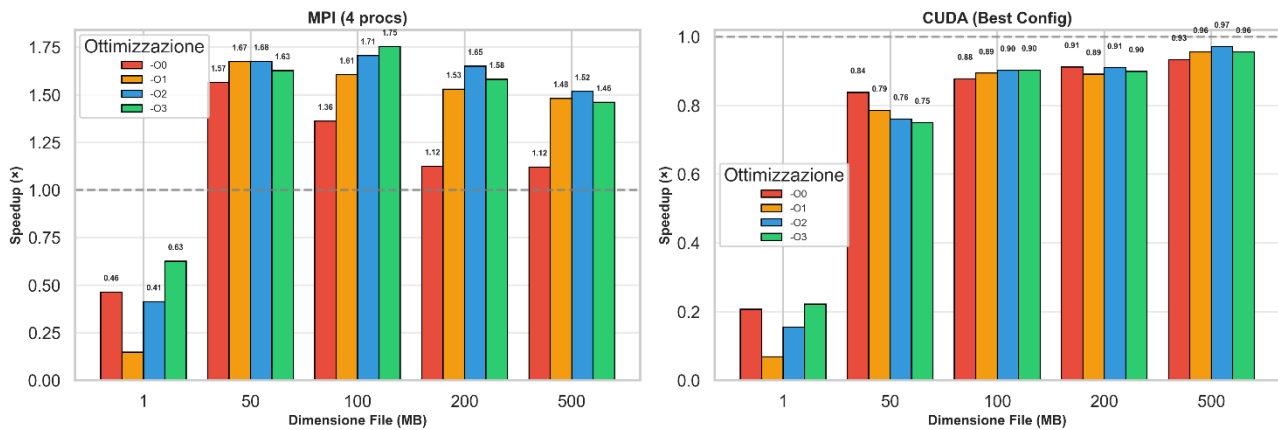


Figura 24: Confronto dello speedup di MPI (4 processi) e CUDA (BS=128/256) rispetto alla baseline sequenziale per ciascun livello di ottimizzazione del compilatore. MPI mostra speedup crescenti da 1.36x (-O0) a 1.75x (-O3), mentre CUDA rimane stabile a 0.88-0.90x, confermando che l'overhead di trasferimento GPU è indipendente dalle ottimizzazioni del codice CPU.

La Figura 24 confronta lo speedup di MPI (4 processi) e CUDA (miglior configurazione) rispetto alla baseline sequenziale per ciascun livello di ottimizzazione. Con -O0, MPI su 100 MB ottiene **1.36x** mentre CUDA raggiunge **0.89x**. Con -O1, MPI migliora a **1.61x** e CUDA a **0.90x**. Con -O2, MPI tocca **1.71x** e CUDA **0.90x**. Infine, con -O3, MPI raggiunge **1.75x** mentre CUDA arriva a **0.90x**. Questa tendenza conferma che l'ottimizzazione del compilatore beneficia proporzionalmente tutte le implementazioni, ma non altera la gerarchia prestazionale: MPI resta costantemente superiore, mentre CUDA rimane leggermente inferiore al sequenziale. Il fatto che CUDA mantenga uno speedup stabile di **~0.90x** su tutte le ottimizzazioni suggerisce che l'overhead di trasferimento GPU, essendo fisso e non legato al codice compilato, domina le prestazioni indipendentemente dalle ottimizzazioni applicate al codice CPU.

5.3.7 Comportamento su Diverse Dimensioni di Input

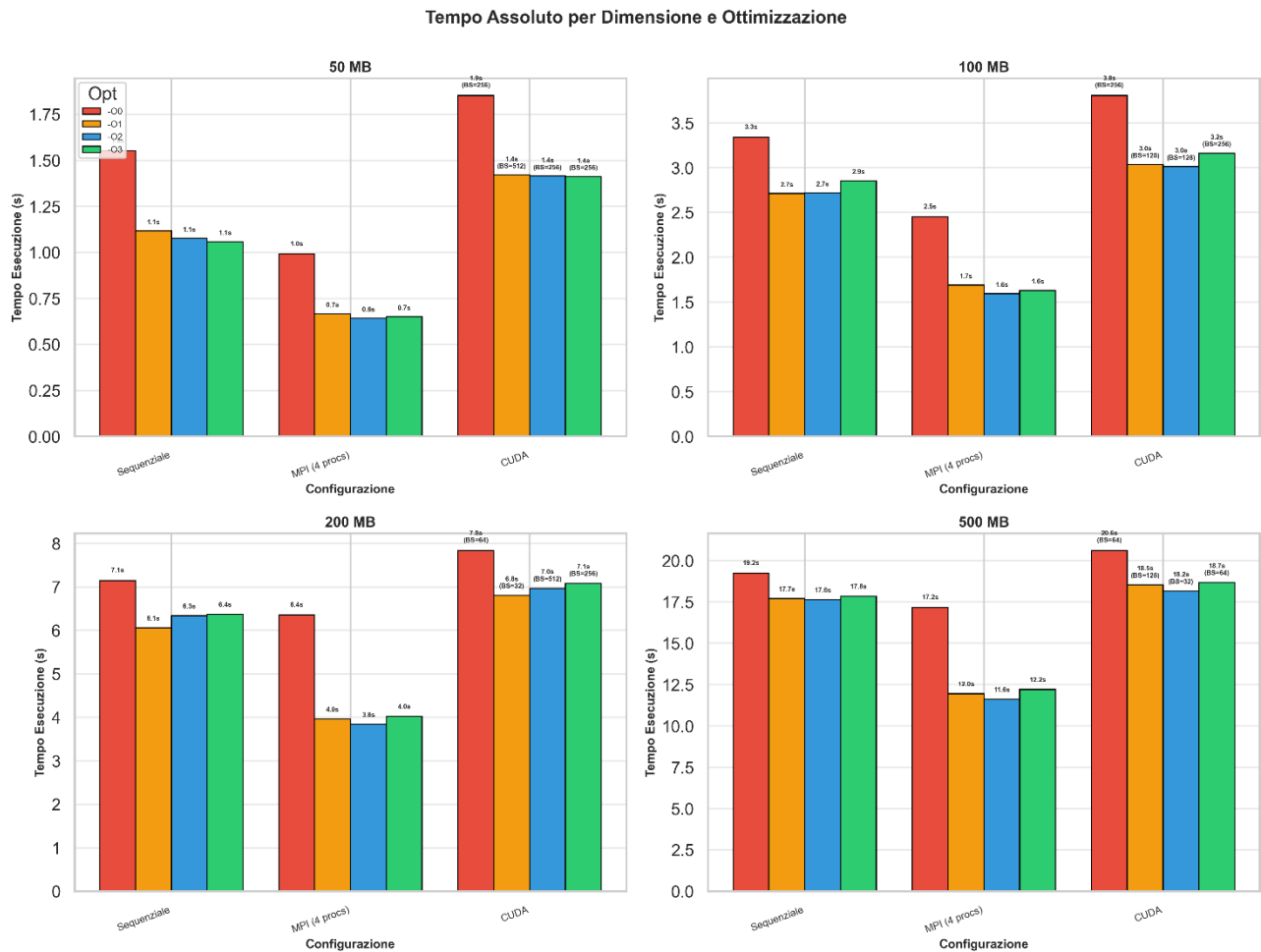


Figura 25: Tempi assoluti di esecuzione per Sequenziale, MPI (4 procs) e CUDA su quattro dimensioni di input (50, 100, 200, 500 MB) con tutte le ottimizzazioni. La crescita quasi lineare con la dimensione è coerente con la complessità $O(N \log N)$ dove il fattore logaritmico cresce lentamente. CUDA mostra pendenza leggermente superiore al sequenziale, indicando che l'overhead di trasferimento GPU scala proporzionalmente ai dati.

La Figura 25 presenta i tempi assoluti di esecuzione per tutte le configurazioni su quattro dimensioni di input (50, 100, 200, 500 MB). Per 50 MB, la versione sequenziale con -O0 richiede **1.55 secondi**, scendendo a **1.06 secondi** con -O3 (31% di miglioramento). MPI (4 processi) riduce ulteriormente a **0.65 secondi** (-O3), mentre CUDA impiega **1.41 secondi**. Su 100 MB, il sequenziale passa da **3.34 secondi** (-O0) a **2.86 secondi** (-O3), MPI a **1.63 secondi** e CUDA a **3.16 secondi**. Per 200 MB, i tempi sono rispettivamente **7.15 secondi** (seq, -O0), **6.37 secondi** (seq, -O3), **4.03 secondi** (MPI) e **7.08 secondi** (CUDA). Infine, su 500 MB, sequenziale -O0 richiede **19.23 secondi**, -O3 **17.85 secondi**, MPI **12.22 secondi** e CUDA **19.06 secondi**. Il grafico evidenzia una crescita quasi lineare dei tempi con la dimensione dell'input per tutte le configurazioni.

5.3.8 Indipendenza dell'Efficienza dalle Ottimizzazioni

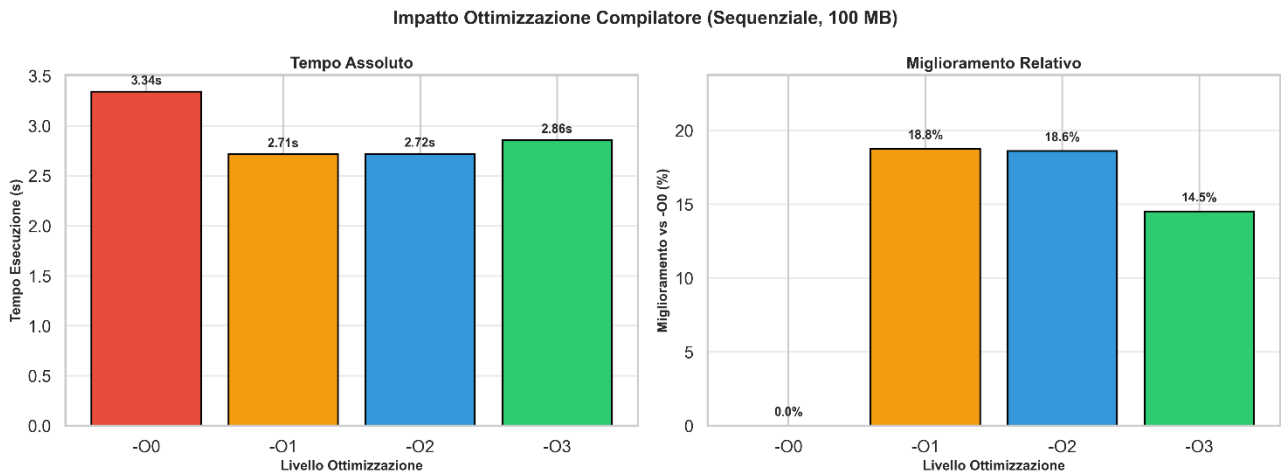


Figura 26: Panoramica completa dell'efficienza parallela MPI al variare di numero processi (2, 4), dimensione input (1-500 MB) e ottimizzazione (-O0 a -O3). Il grafico conferma che l'efficienza è dominata dalla struttura algoritmica piuttosto che dalle ottimizzazioni del compilatore: le quattro curve per ciascuna configurazione sono quasi sovrapposte.

La Figura 26 analizza l'efficacia delle diverse ottimizzazioni del compilatore su un input di 100 MB. Partendo da una baseline non ottimizzata (-O0) di **3.34 secondi**, l'attivazione di -O1 riduce drasticamente il tempo a **2.71 secondi**, ottenendo un miglioramento del 18.8% grazie a tecniche come il function inlining e la propagazione delle costanti. Il livello -O2 si attesta su prestazioni pressoché identiche (**2.72 secondi**), indicando che si è raggiunto un punto di saturazione dei benefici. Significativa è invece la regressione osservata con -O3, che sale a **2.86 secondi** riducendo il guadagno relativo al 14.5%. Questo comportamento controintuitivo è attribuibile alla natura *memory-bound* dell'algoritmo di Manber-Myers: le ottimizzazioni aggressive di -O3 aumentano la dimensione del codice binario. Questo eccesso di istruzioni satura la cache e aumenta la pressione sui registri, finendo per degradare le prestazioni rispetto alla configurazione ottimale -O1/-O2.

5.4 Discussione dei Risultati e Conclusioni

Il presente lavoro ha esplorato l'applicazione di due paradigmi distinti, MPI per architetture a memoria distribuita e CUDA per acceleratori GPU, sull'algoritmo di Manber-Myers. I risultati sperimentali, ottenuti su dataset crescenti da 1 MB a 500 MB, evidenziano chiaramente come la natura dell'algoritmo influenzi profondamente l'efficacia delle strategie di parallelizzazione.

L'implementazione MPI si è dimostrata l'approccio vincente, raggiungendo speedup compresi tra **1.46×** e **1.75×** con 4 processi su input medio-grandi. Questo successo deriva dalla capacità di attaccare direttamente il principale collo di bottiglia computazionale: l'ordinamento dei suffissi nella fase di costruzione del Suffix Array, che rappresenta l'**82%** del tempo totale. Distribuendo questa fase su più processori, si ottengono benefici significativi. Tuttavia, l'efficienza parallela osservata (**40-44%** con $P=4$) evidenzia i limiti intrinseci dell'approccio: la necessità di un merge centralizzato sul Rank 0, che costituisce una frazione seriale del **20-22%**, e l'overhead di comunicazione (gather del testo e dei risultati parziali) limitano lo speedup massimo, causando rendimenti decrescenti all'aumentare dei processi.

Al contrario, l'implementazione CUDA ha prodotto risultati deludenti, con speedup tra **0.75×** e **0.94×**, risultando sistematicamente più lenta della versione sequenziale. La causa risiede nella scelta progettuale di accelerare solo la fase di ricerca della Longest Repeated Substring (LRS). Poiché questa fase costituisce meno del 2% del tempo totale, il margine di miglioramento teorico è trascurabile. Nella pratica, l'overhead di trasferimento dati host-device neutralizza il guadagno computazionale della riduzione parallela, introducendo una penalità netta. Anche l'analisi del block size (variazioni $\pm 2\text{-}3\%$) conferma l'irrelevanza dei parametri di tuning su una fase così marginale. Per ottenere un'implementazione GPU efficace sarebbe necessario parallelizzare le fasi dominanti (SA e LCP), affrontando però sfide algoritmiche notevoli legate agli accessi irregolari alla memoria.

L'analisi delle ottimizzazioni del compilatore conferma ulteriormente questa diagnosi: il livello -O3 mostra benefici tangibili (**14-19%** di miglioramento) proprio perché agisce sulle fasi dominanti (SA e LCP) che occupano oltre il 95% del tempo, a differenza dell'approccio CUDA parziale.

Il confronto diretto quantitativo sancisce il divario tra i due approcci: su input di 100 MB, MPI riduce il tempo di esecuzione a **1.63 secondi** (rispetto ai **2.86** sequenziali), mentre CUDA lo aumenta a **3.16 secondi**. Analogamente, su 500 MB, MPI impiega **12.22 secondi** contro i **19.06 secondi** di CUDA (**17.85** sequenziali). In conclusione, i dati confermano che per la costruzione del Suffix Array su stringhe di dimensioni medio-grandi, un cluster di CPU multi-core con MPI rappresenta una soluzione decisamente più efficiente rispetto a sistemi GPU dedicati.

Bibliografia

1. **Manber, U., & Myers, G.** (1993). Suffix arrays: a new method for on-line string searches. *SIAM Journal on Computing*, 22(5), 935-948. *(Riferimento principale per l'algoritmo di costruzione del Suffix Array utilizzato nel progetto).*
2. **Kasai, T., Lee, G., Arimura, H., Arikawa, S., & Park, K.** (2001). Linear-time longest-common-prefix computation in suffix arrays and its applications. In *Proceedings of the 12th Annual Symposium on Combinatorial Pattern Matching (CPM 2001)* (pp. 181-192). Springer, Berlin, Heidelberg. *(Riferimento per l'algoritmo lineare di calcolo dell'LCP Array).*
3. **Vigna, S.** (2016). An experimental exploration of Marsaglia's xorshift generators, scrambled. *ACM Transactions on Mathematical Software (TOMS)*, 42(4), Article 30. *(Fonte per l'algoritmo di generazione pseudo-casuale xorshift64 utilizzato per il dataset).*
4. **Marsaglia, G.** (2003). Xorshift RNGs. *Journal of Statistical Software*, 8(14), 1-6. *(Introduzione originale della famiglia di generatori xorshift).*